



Companion to Chaos Engineering (Pawlikowski) — Complete Study Guide - Claude. This is the simplified working edition of Labs 0–22.

Start here

1. Complete **Lab 0** once. All commands run in **Bash inside the disposable Linux VM**, except browser-console JavaScript and laptop SSH forwarding.
2. Open one lab. Complete its **core experiment** and recovery check. Optional depth is a separate session.
3. Record a prediction before injection. Compare **baseline → fault → recovery** using the same probe.
4. Stop if setup does not pass its check. A broken setup is not evidence of a resilience failure.
5. Write one explanation of the result before continuing.

Suggested first session: Lab 0 → Lab 1 → Lab 2. Next: 3–4; then 5–11; then 12–14; then 15–20; finish with 21–22.

Dependencies: 6 uses 5; 8 uses 7 only for theory; 9 reuses 5; 11 uses 10; 16–18 use 15; 19 creates a separate cluster. Every other lab starts from Lab 0. The sequence deliberately keeps Linux, Docker and Kubernetes experiments separate.

Scope and reliability

Supplementary explanation: These are modernized teaching experiments inspired by the book, not claims that every command reproduces its historical output. The short core paths isolate one mechanism; optional exercises retain the wider concepts.

The target is **Ubuntu Server 24.04 LTS, Bash, systemd, rootful Docker and cgroup v2**. Other environments need adaptation. Use a VM snapshot before a session. Do not paste these commands into your laptop shell, a work host or a shared cluster. Infrastructure failure experiments were not executed during this document review; the isolated JVM teaching fixture was exercised locally as described in the review notes.

Expected results are hypotheses to check. Timings, throughput, process exit codes through wrappers, scheduler placement and HTTP failure counts depend on the recorded versions and environment. A successful injection does not imply a successful application outcome. Zero observed failures is evidence only for the probe rate and observation window used.

Recovery rule: keep a second VM terminal open. A shell trap handles normal exit and signals such as Ctrl-C; it cannot recover from SIGKILL, a lost VM or a host crash. Use the explicit cleanup command or revert the snapshot in those cases. Run one fault at a time.

Small lab journal

Lab / date / versions:

Prediction: If ___, then metric ___ stays below/above ___ for ___ seconds.

Baseline: sample count, HTTP status/exit status, latency or throughput.

Fault: exact target, command, duration and evidence the injection worked.

Result: observed numbers; supported / refuted / inconclusive.
Recovery: cleanup command and successful baseline probe afterward.
Why: mechanism, confounders and one follow-up.

Measurement convention: `curl` reports transport failures separately from HTTP status. A fast 500 response is not successful service. For `ab`, inspect both `Failed requests` and `Non-2xx responses`; `-l` ignores variable response length, not status or application correctness. Do not call the maximum of 20 samples a trustworthy p99.

Lab 0 — Prepare one disposable VM



Theory before you start

- **Concept:** Chaos engineering tests whether a system still delivers useful service under a controlled fault. The hypothesis predicts an observable outcome; the baseline records normal behaviour for comparison. The blast radius includes everything the fault could affect, including shared dependencies. A useful experiment changes one main condition so you can explain why the result changed.
- **Mechanism:** A disposable Linux virtual machine supplies the kernel features used throughout these labs and separates the experiments from your normal working environment. Version records and a fixed setup reduce accidental differences between runs. Snapshots provide a recovery fallback; explicit cleanup removes the intended fault. Checking baseline → fault → recovery helps distinguish the fault's effect from an already-broken setup.
- **Read the result:** Follow observability → baseline → hypothesis → injection → analysis, then verify recovery. A failed setup is not a resilience finding, and one successful run supports only the conditions actually tested.

Study-guide basis: Ch. 1 §§1.1–1.3; Ch. 2 §2.5; Appendix A.

Goal: Create a known environment and a version record.

Before you start: A fresh Ubuntu Server 24.04 VM; sudo access.

Time: 45–90 min (estimate; downloads excluded).

Core experiment

1. Allocate the VM

Use 4 vCPU, 12 GB RAM and 60 GB disk as a starting point. Lab 19 may need more memory depending on image versions; delete the earlier cluster before creating HA. Choose the native CPU architecture of your hypervisor. Take snapshot `00-clean`.

2. Install the base tools

```

sudo apt-get update
sudo apt-get install -y ca-certificates curl git jq python3 python3-venv \
python3-redis redis-server build-essential nginx apache2-utils sysstat \
stress-ng fio strace libseccomp-dev libcap2-bin util-linux iproute2 \
iptables tcpdump bc unzip openjdk-17-jdk-headless docker.io docker-
compose-v2
sudo systemctl enable --now docker
sudo systemctl disable --now nginx redis-server
sudo usermod -aG docker "$USER"
mkdir -p ~/labs

```

Log out of the VM and log back in so Docker group membership takes effect. Docker group membership grants powerful host access; this account is for the disposable VM.

3. Check the environment

```

uname -m
stat -fc %T /sys/fs/cgroup
systemd --version | head -1
docker info --format 'driver={{.CgroupDriver}} cgroups={{.CgroupVersion}}'
docker run --rm hello-world
docker compose version
df -h /

```

Checkpoint: `cgroup2fs`, Docker cgroup version `2`, a successful hello-world run and a working Compose plugin. If a check fails, fix setup before proceeding. The root `cgroup.subtree_control` list need not contain every controller; check the actual workload's cgroup later.

4. Save the book sources and versions

```

git clone https://github.com/seeker89/chaos-engineering-book.git
~/labs/book
git -C ~/labs/book checkout 3e3ee64db71f51a5e9f79af8562dd4aa913a0e71
git -C ~/labs/book rev-parse HEAD | tee ~/labs/book-commit.txt
{ uname -a; systemd --version; docker version; docker compose version;
python3 --version; java -version; strace --version; } >
~/labs/versions.txt 2>&1

```

The source commit in `book-commit.txt` identifies what you actually use. Image tags are convenient but mutable: after pulling a lab image, record `docker image inspect IMAGE --format '{{json .RepoDigests}}'`. Do not silently substitute versions after a failure.

5. Save reusable helpers

Save as `~/labs/common.sh`; run `source ~/labs/common.sh` in each new VM terminal.

```

cg() {
    local p rel
    p=$(docker inspect -f '{{.State.Pid}}' "$1") || return
    [ "$p" -gt 1 ] || { echo 'Container must be running' >&2; return 1; }
    rel=$(sudo awk -F: ' $1=="0" {print $3}' "/proc/$p/cgroup") || return
    printf '/sys/fs/cgroup%s\n' "$rel"
}
measure() {
    local url=$1 count=${2:-10}
    for ((i=1;i<=count;i++)); do
        curl --noproxy '*' -sS --connect-timeout 2 --max-time 10 -o /dev/null \
            -w '%{http_code} %{time_total}\n' "$url"
    done
}

```

Done when: all setup checks pass and `01-tools` snapshot exists.

Cleanup / recovery: failed installation → correct the package error or restore `00-clean`. Do not continue with half-installed tools.

If it differs: check the Ubuntu release, architecture, apt error and Docker daemon status before installing substitutes.

Why: a recorded environment separates application findings from setup and version differences.

- Optional depth — continue after the core works

Optional depth: BCC tools require matching kernel support: install `bpffcc-tools bpftrace linux-headers-$(uname -r)` only for the tracing extensions. If unavailable, the core experiments still use standard `/proc`, `vmstat`, `iostat` and `strace`.

Lab 1 — Compare a refused connection with a silent dependency



Theory before you start

- **Concept:** A dependency can fail quickly, respond too slowly, or never respond within a useful time. These are different conditions for the caller. An exception handler only runs after an operation returns or raises; it cannot rescue a call that remains blocked. While waiting, requests may occupy worker slots or connections, reducing the application's ability to serve other users.
- **Mechanism:** In this lab, REJECT returns a TCP refusal while DROP silently discards traffic, leaving the client waiting for a response or timeout. A connection timeout limits connection establishment; a read timeout limits waiting on an established connection. The external watchdog bounds the demonstration. A whole-request deadline must also account for multiple dependency calls and retries, rather than restarting the full waiting budget at each step.

- **Read the result:** Compare elapsed time and outcome, not just whether an error was printed. The external watchdog limits the experiment; the application's own timeout prevents unbounded waiting at that boundary.

Study-guide basis: Ch. 1 §1.5.

Goal: Observe why a timeout is needed even when errors are handled.

Before you start: Lab 0. No other service should use Redis in this VM.

Time: 25–35 min (estimate; downloads excluded).

Core experiment

1. Prepare a tiny client

```
sudo systemctl start redis-server
redis-cli ping
mkdir -p ~/labs/lab01 && cd ~/labs/lab01
cat > client.py <<'PY'
import os, time, redis
from redis.retry import Retry
from redis.backoff import NoBackoff
t = float(os.environ['TIMEOUT']) if 'TIMEOUT' in os.environ else None
c = redis.Redis(host='127.0.0.1', port=6379,
               socket_connect_timeout=t, socket_timeout=t,
               retry=Retry(NoBackoff(), 0))
start = time.monotonic()
try:
    print('OK', c.ping())
except redis.exceptions.RedisError as exc:
    print('DEGRADED', type(exc).__name__)
print(f'elapsed={time.monotonic()-start:.3f}s')
PY
python3 client.py
```

Checkpoint: PONG, then OK True. Retries are disabled so a retry policy does not disguise the timeout.

2. Create a bounded comparison

Save as `run.sh` in this folder. It owns exactly one OUTPUT rule, identified by a lab comment; it never flushes firewall tables.

```
#!/usr/bin/env bash
set -u
rule=(-p tcp -d 127.0.0.1 --dport 6379 -m comment --comment ce-lab01)
cleanup() {
    sudo iptables -D OUTPUT "${rule[@]}" -j DROP 2>/dev/null || true
    sudo iptables -D OUTPUT "${rule[@]}" -j REJECT --reject-with tcp-reset
    2>/dev/null || true
}
```

```

}
trap cleanup EXIT
trap 'exit 130' INT TERM
sudo -v || exit 1
cleanup
echo 'REJECT'
sudo iptables -I OUTPUT 1 "${rule[@]}" -j REJECT --reject-with tcp-reset ||
exit 1
timeout 5s python3 client.py; echo "exit=$?"
cleanup
echo 'DROP, no application timeout'
sudo iptables -I OUTPUT 1 "${rule[@]}" -j DROP || exit 1
sudo iptables -nvL OUTPUT --line-numbers
timeout 5s python3 client.py; echo "exit=$?"
echo 'DROP, 0.5-second application timeout'
TIMEOUT=0.5 timeout 5s python3 client.py; echo "exit=$?"

```

3. Predict, run, recover

Predict which case reaches the exception handler promptly. Run `bash run.sh`, then `python3 client.py` and `sudo iptables -S OUTPUT | grep ce-lab01`. No matching rule should remain (grep exit 1 is expected).

Expected evidence: REJECT normally gives a fast connection error. DROP without an application timeout is bounded by the external `timeout` command (normally exit 124). With the application timeout, the exception handler should run near 0.5 seconds; record the actual elapsed time.

Cleanup / recovery: normally automatic. If the script was killed, run these exact removals in the second terminal:

```

sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6379 -m comment --
comment ce-lab01 -j DROP
sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6379 -m comment --
comment ce-lab01 -j REJECT --reject-with tcp-reset

```

An absent-rule error is harmless. Repeat the client; it must work before finishing.

Why: an explicit refusal supplies an error; silence may leave the client waiting. The external watchdog bounds the experiment; the application timeout is the actual fix.

Done when: you have three outcomes, elapsed times and a successful recovery probe.

If it differs: check Redis first; inspect rule counters; verify no retry wrapper was reintroduced.

- Optional depth — continue after the core works

Optional depth: repeat using a persistent connection and `GET` every 200 ms. This distinguishes a blocked read from a new connection attempt. Explain why socket timeouts do not necessarily impose one total deadline over a multi-call request.

Lab 2 — Identify signals and memory-limit kills



Theory before you start

- **Concept:** Signals notify processes of events or request termination. SIGTERM gives an application an opportunity to handle shutdown and release resources; SIGKILL cannot be caught or ignored. Bash normally reports a fatal signal as 128 plus its number: 143 for SIGTERM and 137 for SIGKILL. These values identify how execution ended, not who sent the signal or why.
- **Mechanism:** A cgroup memory limit constrains a group independently of total free host memory. When its memory demand cannot be satisfied through reclaim, an out-of-memory (OOM) kill may terminate a member. Separately, virtual address space describes mappings, while resident set size (RSS) describes pages currently in RAM. The mmap comparison delays touching pages so you can observe these two quantities diverge.
- **Read the result:** Exit 137 alone does not establish an out-of-memory kill. Correlate the exit with kernel/unit logs and limit evidence; compare virtual size with RSS.

Study-guide basis: Ch. 2 §2.3; supplementary memory clarification. Signal exit status

Goal: Use evidence beyond an exit code.

Before you start: Lab 0; keep journal logs instead of clearing dmesg.

Time: 25–40 min (estimate; downloads excluded).

Core experiment

1. Compare two signals

Run in an interactive Bash terminal without `set -e`:

```
sleep 60 & victim=$!; kill -TERM "$victim"; wait "$victim"; echo "TERM=$?"
sleep 60 & victim=$!; kill -KILL "$victim"; wait "$victim"; echo "KILL=$?"
python3 -c 'import os,signal; os.kill(os.getpid(),signal.SIGFPE)'; echo
"FPE=$?"
```

Bash normally reports 143, 137 and 136. `128 + signal` identifies the signal, not the reason it was sent. A wrapper such as `systemd-run` may report status differently.

2. Cause a cgroup-bounded OOM

```
since=$(date --iso-8601=seconds)
sudo systemd-run --unit=ce-lab02 --wait --collect \
  -p MemoryMax=128M -p MemorySwapMax=0 -p RuntimeMaxSec=20s \
  /usr/bin/python3 -c 'a=[]
while True: a.append(b"x"*1000000)'
sudo journalctl -u ce-lab02 --since "$since" --no-pager
```

```
sudo journalctl -k --since "$since" --no-pager | grep -iE 'oom|killed process'
```

Prediction: the workload exceeds its 128 MiB cgroup limit before the 20-second runtime bound. Confirm using unit and kernel evidence. If userspace `systemd-oomd` acted, identify it with `journalctl -u systemd-oomd`; do not call every SIGKILL a kernel OOM kill.

3. Separate virtual allocation from touched memory

```
mkdir -p ~/labs/lab02 && cd ~/labs/lab02
cat > pages.py <<'PY'
import mmap, os, time
n=256*1024*1024
m=mmap.mmap(-1,n)
print('PID',os.getpid(),'mapped, not yet touched',flush=True)
time.sleep(10)
for offset in range(0,n,mmap.PAGESIZE): m[offset]=1
print('touched every page',flush=True)
time.sleep(10)
PY
python3 pages.py & victim=$!
for i in $(seq 1 20); do ps -p "$victim" -o pid,vsz,rss,comm; sleep 1; done
wait "$victim"
```

Expected evidence: virtual size grows on mapping; RSS grows after writes. Allocator behaviour makes `bytearray()` an unreliable substitute for this comparison. Python mmap documentation.

Cleanup / recovery: `sudo systemctl stop ce-lab02.service` if still active; `kill "$victim"` only if that recorded process is still running. The runtime bound and finite page example also terminate independently.

Done when: your journal explains why 137 alone is insufficient evidence and shows mapping versus residency.

If it differs: compare timestamps, scope and kernel logs; a missing log line is not proof that no memory kill occurred.

- Optional depth — continue after the core works

Optional depth: use `cat /proc/$$/oom_score /proc/$$/oom_score_adj`. In a separate 128 MiB bounded unit, compare two allocators with different `oom_score_adj`; treat victim selection as a measured result. An integer division-by-zero C expression has undefined behaviour and is not a portable way to promise SIGFPE. Distinguish a deliberately sent SIGFPE from an architecture-specific hardware arithmetic trap.

Lab 3 — See a restart limit stop recovery



Theory before you start

- **Concept:** A process supervisor and a load balancer protect different parts of availability. systemd can restart a crashed backend; NGINX can direct requests toward other usable backends. Redundancy helps only while enough working capacity remains. A successful request through the proxy can therefore conceal a failed instance, and a running process still needs to serve correct responses.
- **Mechanism:** systemd checks both the restart policy and the permitted number of starts within a configured interval. Rapid repeated crashes can exhaust this allowance and leave a unit in start-limit-hit despite Restart=always. The remaining backend may carry requests until it reaches its own limit. Comparing isolated crashes with repeated crashes exposes how restart timing and backend availability interact.
- **Read the result:** Track backend state, restart count and client responses separately. Disabling the start limit removes one recovery barrier, but it neither repairs the crash nor guarantees uninterrupted service.

Study-guide basis: Ch. 2 §§2.4–2.6.

Goal: Separate a supervisor restart policy from service availability.

Before you start: Lab 0; ports 8001–8003 free.

Time: 35–50 min (estimate; downloads excluded).

Core experiment

1. Create two explicitly configured services

```

sudo mkdir -p /srv/ce-lab03
echo 'lab03 OK' | sudo tee /srv/ce-lab03/index.html
for pair in a:8001 b:8002; do
    name=${pair%:*}; port=${pair#*:}
    sudo tee "/etc/systemd/system/ce-lab03-$name.service" >/dev/null <<UNIT
[Unit]
Description=Chaos lab 03 $name
StartLimitIntervalSec=20
StartLimitBurst=4
[Service]
ExecStart=/usr/bin/python3 -m http.server $port --bind 127.0.0.1 --
directory /srv/ce-lab03
Restart=always
RestartSec=200ms
UNIT
done
sudo systemctl daemon-reload
sudo systemctl start ce-lab03-a ce-lab03-b

```

Create the proxy:

```

sudo tee /etc/nginx/conf.d/ce-lab03.conf >/dev/null <<'NGX'
upstream ce_lab03 { server 127.0.0.1:8001 max_fails=1 fail_timeout=1s;
server 127.0.0.1:8002 max_fails=1 fail_timeout=1s; }
server { listen 127.0.0.1:8003; location / { proxy_pass http://ce_lab03; }
}
NGX
sudo nginx -t && sudo systemctl restart nginx
source ~/labs/common.sh
measure http://127.0.0.1:8003/ 10

```

Checkpoint: ten 200 responses. In a second terminal run `watch -n 1 'systemctl is-active ce-lab03-a ce-lab03-b'`.

2. Kill A once; then repeat quickly

```

sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a
sleep 1
curl -fsS --max-time 3 http://127.0.0.1:8003/
for i in $(seq 1 6); do
    sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a || true
    sleep 1
done
systemctl show ce-lab03-a -p ActiveState -p Result -p NRestarts
sudo journalctl -u ce-lab03-a -n 15 --no-pager

```

Prediction: A eventually hits its start limit; B can keep the proxy responding. Do not infer that A recovered from a successful proxy probe.

3. Repeat against B while measuring

In terminal 2, record 30 seconds of HTTP codes using `for i in $(seq 1 60); do curl -s --max-time 1 -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8003/; sleep 0.5; done`. In terminal 1, repeat the six-kill loop with `ce-lab03-b`.

Expected evidence: both units can hit `start-limit-hit`; the proxy then loses usable backends. A single successful run does not establish zero downtime under other timings.

4. Recover and test one change

```

sudo systemctl reset-failed ce-lab03-a ce-lab03-b
sudo systemctl start ce-lab03-a ce-lab03-b
curl -fsS --max-time 3 http://127.0.0.1:8003/

```

For an optional A/B comparison, add `[Unit] StartLimitIntervalSec=0` via `sudo systemctl edit ce-lab03-a`; reload, restart A and repeat the same injection. This removes the give-up threshold; it does not fix the cause of crashes.

Cleanup / recovery: stop the two exact units, remove only their files and drop-in directories, remove `/etc/nginx/conf.d/ce-lab03.conf`, daemon-reload and reload nginx:

```
sudo systemctl stop ce-lab03-a ce-lab03-b
sudo rm -f /etc/systemd/system/ce-lab03-a.service /etc/systemd/system/ce-lab03-b.service /etc/nginx/conf.d/ce-lab03.conf
sudo rm -rf /etc/systemd/system/ce-lab03-a.service.d /etc/systemd/system/ce-lab03-b.service.d
sudo systemctl daemon-reload
sudo nginx -t && sudo systemctl reload nginx
```

Done when: you distinguish backend state, restart count and end-user response.

If it differs: inspect effective start-limit values and the journal rather than relying on distribution defaults.

- Optional depth — continue after the core works

Optional depth: test `Restart=on-failure` with SIGTERM versus SIGKILL. On systemd 254+, compare `RestartSteps` and `RestartMaxDelaySec` under repeated crashes; backoff is not identical to Kubernetes CrashLoopBackOff. Compare a process-name grep with exact systemd unit targeting without sending signals.

Lab 4 — Find CPU contention with two measurements



Theory before you start

- **Concept:** USE means utilization, saturation and errors. CPU utilization measures time spent working; saturation means runnable tasks must wait for execution. A CPU can be fully busy doing useful batch work without violating an application's target. Contention becomes important when that waiting increases the target workload's completion time. Measure the application effect alongside the resource signal.
- **Mechanism:** CPU affinity places both workloads on the same CPU, making competition easier to observe. A quota limits the neighbour's CPU-time budget per period; after consuming it, the group is throttled until its budget renews. Nice changes relative scheduling preference rather than imposing a hard cap. Pressure Stall Information (PSI) records resource waiting, while throttling counters reveal enforcement of the quota.
- **Read the result:** Compare task duration with waiting and throttling counters. A quota may help the target by constraining its neighbour; the size of that improvement depends on the workload.

Study-guide basis: Ch. 3 §§3.1–3.3; Ch. 5 cgroups.

Goal: Use utilization and waiting together; compare priority with a hard quota.

Before you start: Lab 0. Use one chosen CPU in the VM.

Time: 30–45 min (estimate; downloads excluded).

Core experiment

1. Create a finite workload

```
mkdir -p ~/labs/lab04 && cd ~/labs/lab04
cat > work.py <<'PY'
import time
for i in range(24):
    start=time.monotonic()
    sum(j*j for j in range(5000000))
    print(i,round(time.monotonic()-start,4),flush=True)
PY
taskset -pc $$
```

Choose an allowed CPU from the printed list; the following uses CPU 0. Change every `-c 0` if CPU 0 is unavailable.

2. Record baseline

Run `taskset -c 0 python3 work.py | tee baseline.txt` three times. Record the range of iteration durations.

3. Add one bounded neighbour

```
sudo systemd-run --unit=ce-lab04 --collect -p RuntimeMaxSec=45s \
    taskset -c 0 stress-ng --cpu 1 --timeout 40s
taskset -c 0 python3 work.py | tee loaded.txt
cat /proc/pressure/cpu
vmstat 1 5
sudo systemctl stop ce-lab04
```

Prediction: iteration time increases while a runnable neighbour competes for the same CPU. CPU PSI is a waiting signal, not utilization. `avg10` is a smoothed average; it is not an exact rectangular 10-second sample.

4. Limit the neighbour and repeat

```
sudo systemd-run --unit=ce-lab04 --collect -p CPUQuota=20% -p
RuntimeMaxSec=45s \
    taskset -c 0 stress-ng --cpu 1 --timeout 40s
taskset -c 0 python3 work.py | tee quota.txt
path=$(systemctl show ce-lab04 -p ControlGroup --value)
sudo cat "/sys/fs/cgroup$path/cpu.max" "/sys/fs/cgroup$path/cpu.stat"
sudo systemctl stop ce-lab04
```

Expected evidence: the neighbour is capped at 20% of one CPU; throttling counters should rise. The target may improve, but exact latency improvement is measured, not guaranteed. Repeat with `nice -n 19` in the neighbour command to contrast relative priority and quota.

Cleanup / recovery: `sudo systemctl stop ce-lab04`; repeat the baseline and check for recovery. Wait for the old transient unit to disappear before reusing its name.

Done when: you have baseline, contention and quota measurements plus a kernel counter explaining the change.

If it differs: check CPU affinity, existing VM load and whether the target finished before the neighbour started.

- Optional depth — continue after the core works

Optional depth — USE resource tour: inspect `free -h / /proc/pressure/memory` for RAM and `iostat -xz 1 / /proc/pressure/io` for I/O. Saturation and `%util` are not interchangeable; parallel devices can stay busy without being at maximum throughput. Use `sar -n DEV,TCP,ETCP 1` for networking and BCC `execsnoop-bpfcc` / `opensnoop-bpfcc` for process and file activity. Bound every load to 30 seconds and allocate memory inside a capped unit.

Optional depth — slow start: delay the neighbour by 20 seconds and collect at least 60 seconds of repeated target iterations; a 10-second window would miss it. Compare `CPUWeight=100` and `900` only for contending sibling cgroups, then compare quota.

Optional depth — Prometheus: Node Exporter supplies `node_cpu_seconds_total`, `node_memory_MemAvailable_bytes` and, where supported, `node_pressure_cpu_waiting_seconds_total`. Use `rate(node_pressure_cpu_waiting_seconds_total[1m])` as a waiting fraction and `quantile_over_time(0.95, rate(node_pressure_cpu_waiting_seconds_total[1m])[30m:15s])` to summarize a 30-minute window. Collect that window before interpreting it. Metrics availability and scrape target health must be checked first.

Lab 5 — Measure disk contention without guessing capacity



Theory before you start

- **Concept:** Storage has separate capacity and performance limits: how much data fits, and how quickly operations finish. Throughput is data transferred per second; latency is time per operation. Competing I/O can increase queues and delay a database even when plenty of space remains. Application impact depends on whether the tested request actually needs that storage.
- **Mechanism:** The finite fio writer competes with the WordPress/MySQL stack when they share backing storage. Calibration establishes throughput for this file, access pattern and VM; changing block size or read/write mix changes the meaning of that number. Direct I/O reduces operating-

system page-cache effects but not every cache. A cached page may show little slowdown even while the writer produces substantial disk activity.

- **Read the result:** Compare successful request throughput, latency and errors alongside I/O activity. Little application impact can mean the tested request barely depends on disk; verify the injected load first.

Study-guide basis: Ch. 3 §3.3.2; Ch. 4 §4.2.1.

Goal: Compare an application baseline with one bounded disk-writing neighbour.

Before you start: Lab 0; 5 GB free in the VM; port 8080 free.

Time: 45–60 min (estimate; downloads excluded).

Core experiment

1. Start WordPress and MySQL

Save as `~/labs/lab05/compose.yaml` (create the directory first):

```
name: ce-lab05
services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: lab-only-root
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: lab-only-wordpress
    volumes: [dbdata:/var/lib/mysql]
    healthcheck:
      test: [CMD-SHELL, 'mysqladmin ping -h 127.0.0.1 --silent']
      interval: 5s
      timeout: 3s
      retries: 30
  wordpress:
    image: wordpress:6-apache
    depends_on:
      db: {condition: service_healthy}
    ports: ['127.0.0.1:8080:80']
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: lab-only-wordpress
      WORDPRESS_DB_NAME: wordpress
    volumes: [wp:/var/www/html]
  volumes: {dbdata: {}, wp: {}}
```

Run `cd ~/labs/lab05 && docker compose up -d`. From your laptop use `ssh -L 8080:127.0.0.1:8080 VM_USER@VM_ADDRESS`, replacing both values. Open

`http://localhost:8080` and complete the WordPress installer with disposable credentials. Always measure that same hostname to avoid redirects.

Checkpoint: `curl -sS --max-time 10 -o /dev/null -w '%{http_code}\n'`
`http://localhost:8080/` returns 200, and Compose shows a healthy database. If not, inspect `docker compose logs --tail=40`.

2. Record the baseline

```
cd ~/labs/lab05
ab -t 15 -c 1 -l http://localhost:8080/ | tee baseline.txt
```

Repeat three times. Record RPS, failed/non-2xx requests and the percentile table. Keep URL, concurrency and duration fixed.

3. Calibrate a bounded file workload

```
fio --name=calibrate --filename="$HOME/labs/lab05/fio.bin" --size=512M \
  --rw=write --bs=1M --direct=1 --time_based --runtime=15 --group_reporting
```

Run `iostat -xz 1` in terminal 2. This is the throughput of this file, filesystem, VM and access pattern; it is not universal disk capacity. Direct I/O reduces page-cache distortion but does not bypass every storage-layer cache.

4. Repeat the probe under the same writer

```
fio --name=neighbour --filename="$HOME/labs/lab05/fio.bin" --size=512M \
  --rw=write --bs=1M --direct=1 --time_based --runtime=25 --group_reporting
> fio.txt &
writer=$!
sleep 3
ab -t 15 -c 1 -l http://localhost:8080/ | tee loaded.txt
wait "$writer"
ab -t 15 -c 1 -l http://localhost:8080/ | tee recovered.txt
```

Prediction: disk contention may raise request latency. A cached/read-heavy WordPress page may barely change; that is a valid limitation, not a failed injection. Verify actual I/O counters before attributing changes.

Cleanup / recovery: wait for the 25-second writer, then `rm -f ~/labs/lab05/fio.bin`. Keep the stack for Labs 6 and 9; otherwise `cd ~/labs/lab05 && docker compose down`. Add `-v` only when intentionally discarding this lab's database and WordPress volume.

Done when: baseline, loaded and recovered results exist, including errors and an I/O observation.

If it differs: confirm the file and Docker volumes use the intended backing storage (`findmnt`, `lsblk`, `docker info`); read CPU usage too. On a VM, host storage contention remains a confounder.

- Optional depth — continue after the core works

Optional depth: use `fio --rate=10m, 25m`, then `50m` only if the calibration can sustain those rates. Record achieved rate, not just configured rate. Compare buffered writes with `--direct=1` and `dd ... conv=fdatasync`; synchronization and bypassing cache answer different questions. For cgroup I/O controls, resolve the real backing block device and verify `io.max` and `io.stat`; do not assume `/dev/vda` or that a cap equals a fixed percentage of disk saturation.

Lab 6 — Delay only database traffic



Theory before you start

- **Concept:** A dynamic web page can require several dependent database exchanges before it can respond. Their waiting times accumulate along the critical path: the sequence that determines completion time. Parallel independent calls overlap, so their delays do not simply add. This is why a small dependency delay can cause a much larger user-visible slowdown, and why call structure matters.
- **Mechanism:** Linux traffic control uses a queueing discipline (qdisc) to schedule packets. A filter selects MySQL-bound traffic leaving WordPress, and netem holds those packets for the configured delay; unmatched traffic uses another band. The static page provides a comparison with little database dependency. Injection is directional and packet-based, so one configured delay is neither a guaranteed round-trip increase nor a delay applied once per SQL query.
- **Read the result:** Compare dynamic and static pages and confirm matching packet counters rise. Extra latency is not simply SQL-query count multiplied by delay: buffering, packetisation, handshakes and parallelism also matter.

Study-guide basis: Ch. 4 §4.2.2.

Goal: Show how repeated dependency calls multiply latency.

Before you start: Lab 5 working; source `~/labs/common.sh`.

Time: 30–45 min (estimate; downloads excluded).

Core experiment

1. Resolve the running container

```
cd ~/labs/lab05
WP=$(docker compose ps -q wordpress)
WP_PID=$(docker inspect -f '{{.State.Pid}}' "$WP")
[ "$WP_PID" -gt 1 ] || { echo 'WordPress is not running'; false; }
net() { [ "$WP_PID" -gt 1 ] || return 1; sudo nsenter --target "$WP_PID" --
```

```
net -- "$@"; }
net ip -4 addr show eth0
net tc qdisc show dev eth0
source ~/labs/common.sh
measure http://localhost:8080/ 10
measure http://localhost:8080/wp-includes/css/dashicons.min.css 10
```

Checkpoint: both URLs return 200; `eth0` is the application's interface. Do not overwrite an existing custom root qdisc. These instructions assume the fresh Docker interface's default qdisc.

2. Add a targeted delay

```
net tc qdisc add dev eth0 root handle 1: prio bands 3 priomap 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1
net tc filter add dev eth0 protocol ip parent 1: prio 1 u32 \
    match ip protocol 6 0xff match ip dport 3306 0xffff flowid 1:1
net tc qdisc add dev eth0 parent 1:1 handle 10: netem delay 10ms
```

Unmatched traffic goes to band 2. The filter delays IPv4 TCP packets destined for MySQL, on WordPress egress only. It does not add 10 ms in both directions.

3. Measure three doses

```
for delay in 1 10 50; do
    net tc qdisc change dev eth0 parent 1:1 handle 10: netem delay
    "${delay}ms"
    echo "delay=${delay}ms"
    measure http://localhost:8080/ 10
    measure http://localhost:8080/wp-includes/css/dashicons.min.css 3
    net tc -s qdisc show dev eth0
done
```

Prediction: dynamic-page latency rises more than one configured delay if it performs sequential database exchanges; the static file changes much less. Packet counters verify the injection. A 1 ms run is a low-dose comparison, not a pure measurement of tool overhead; compare the same qdisc with `delay 0ms` for that.

4. Recover

```
net tc qdisc del dev eth0 root
net tc qdisc show dev eth0
measure http://localhost:8080/ 10
```

Cleanup / recovery: the preceding delete is the manual kill switch. If the container restarted, resolve its PID again before entering its namespace. Keep Lab 5's stack for Lab 9.

Done when: the fault counter increased, the dynamic/static comparison is recorded and the baseline recovered.

If it differs: check HTTP codes, DB port/IP version, container restart, cache behaviour and qdisc counters. Unchanged latency with zero matching packets is an invalid injection.

Why: $\text{extra time} \approx \text{delay} \times \text{sequential critical-path exchanges}$ is an approximation. SQL query count is not exactly TCP packet count or round trips; buffering, ACKs, handshakes and parallel work matter.

- Optional depth — continue after the core works

Optional depth: compare a whole-interface root `netem` delay only after removing this hierarchy; it also delays HTTP responses. Use MySQL `SHOW GLOBAL STATUS LIKE 'Questions'` before and after an isolated page request, subtract the probe/background cost, then compare the slope against the query count. Treat a mismatch as evidence to investigate rather than force-fitting the model. Avoid the old 2-second-per-packet exercise until the shorter series and recovery work.

Lab 7 — Build a small isolated process environment



Theory before you start

- **Concept:** A Linux container combines mechanisms that separate a process's view of the system and constrain its resource use. Containers normally share the host kernel, while a virtual machine has its own guest kernel. Seeing an isolated filesystem or PID list does not imply independent CPU, memory or storage capacity, and does not by itself establish a secure boundary.
- **Mechanism:** chroot changes where filesystem paths beginning with a slash are resolved. Mount and PID namespaces provide separate mount and process-ID views; the same process can have different PIDs inside and outside its namespace. Cgroups account for and limit the workload's consumption. The lab combines these features under a bounded systemd scope, letting you observe visibility and CPU enforcement independently from the host.
- **Read the result:** Identify which mechanism explains each observation: a different filesystem view, PID 1 inside the sandbox, or CPU throttling. This teaching sandbox lacks the full isolation and security configuration of a container runtime.

Study-guide basis: Ch. 5 §§5.4–5.7.

Goal: Separate a filesystem view, a PID namespace and resource controls.

Before you start: Lab 0. This is a teaching sandbox, not a security boundary.

Time: 30–45 min (estimate; downloads excluded).

Core experiment

1. Export a root filesystem

```
mkdir -p ~/labs/lab07/rootfs
cd ~/labs/lab07
cid=$(docker create busybox:1.36)
docker export "$cid" | tar -x -C rootfs
docker rm "$cid"
```

2. Start namespaces under bounded systemd resources

```
sudo systemd-run --unit=ce-lab07 --scope \
  -p MemoryMax=128M -p MemorySwapMax=0 -p TasksMax=50 -p CPUQuota=20% \
  unshare --fork --pid --mount --mount-proc="$HOME/labs/lab07/rootfs/proc" \
  chroot "$HOME/labs/lab07/rootfs" /bin/sh
```

Inside, run `echo $$`, `ps`, and `ls /`. The shell should be PID 1 in its namespace. Exit with `exit`.

3. Read limits from the host while the shell runs

In terminal 2:

```
path=$(systemctl show ce-lab07.scope -p ControlGroup --value)
sudo cat "/sys/fs/cgroup$path/cpu.max" "/sys/fs/cgroup$path/memory.max"
"/sys/fs/cgroup$path/pids.max"
sudo lsns -t pid
```

Prediction: the host sees the sandbox processes, but the sandbox's `/proc` shows only its PID namespace. Root filesystem selection alone does not create this isolation.

4. Test one resource limit

Inside the sandbox run `while ;; do ;; done`. While it runs, terminal 2 reads `cpu.stat` from the same cgroup and `top`. Stop the loop with Ctrl-C; exit the shell.

Expected evidence: CPU quota limits the whole scope to 20% of one CPU and throttling counters increase under sustained demand. This does not isolate disk or network I/O.

Cleanup / recovery: `sudo systemctl stop ce-lab07.scope` from the host stops the sandbox if its shell is unresponsive. The host shell was never moved into the limited cgroup.

Done when: you can identify which feature changes filesystem visibility, PID visibility and resource consumption.

If it differs: verify the mount and PID namespace flags, and that the cgroup paths exist while the scope is running.

- Optional depth — continue after the core works

Optional depth: compare `chroot` without namespaces (do not mount host-wide `proc`); enter a verified sandbox PID with `nsenter --target PID --pid --mount --root`. Select the PID from this unit's process tree, not the first `sh` in the VM. Add `--net` for an empty network namespace, then study a veth pair and host/container IPs before configuring routing. Compare Docker namespace identities with `lsns --task CONTAINER_PID` and `/proc/1/ns/*`.

Optional depth — memory and PIDs: read `memory.events` and `pids.events` after bounded allocators. Use a finite process-creation loop under `TasksMax`, not a recursive fork bomb. A PID limit prevents additional tasks; a memory limit does not substitute for it. Docker, unlike this example, also manages capabilities, seccomp, image layers, networking and lifecycle.

Lab 8 — Test container resource boundaries



Theory before you start

- **Concept:** Container isolation is specific to each resource. Separate process or filesystem views do not automatically reserve CPU, RAM, task slots or shared-volume space. A limit also does not guarantee application progress: preventing one workload from consuming too much can make that workload slower or cause it to fail. The experiment tests those different enforcement outcomes.
- **Mechanism:** A CPU quota throttles execution, a memory limit can lead to allocation failure or an OOM kill, and a PID limit blocks creation of additional tasks, including threads. Shared storage needs separate capacity controls. The two containers here mount one small tmpfs, a memory-backed filesystem: filling it consumes the same space both depend on, regardless of their separate container identities.
- **Read the result:** Match each symptom to its controller counters or shared filesystem. The bounded tmpfs exercise demonstrates shared-space exhaustion; it does not measure block-disk performance or Docker writable-layer quotas.

Study-guide basis: Ch. 5 filesystem sharing and cgroups. cgroup v2 controls

Goal: Observe what CPU, memory, PID and shared-storage limits actually constrain.

Before you start: Lab 0; source `~/labs/common.sh`.

Time: 40–60 min (estimate; downloads excluded).

Core experiment

1. Read a hard CPU cap

```
docker run -d --name ce-lab08-cpu --cpus=0.5 python:3.12-slim \
  python -c 'import time; end=time.monotonic()+30
while time.monotonic()<end: pass'
path=$(cg ce-lab08-cpu)
cat "$path/cpu.max" "$path/cpu.stat"
docker stats --no-stream ce-lab08-cpu
```

```
docker wait ce-lab08-cpu
docker rm ce-lab08-cpu
```

Prediction: CPU use is bounded around half of one CPU, subject to the measurement interval. Inspect quota/period instead of assuming a particular period value.

2. Bound a memory allocator

```
docker run --name ce-lab08-mem --memory=64m --memory-swap=64m python:3.12-slim \
  python -c 'a=[]
while True: a.append(b"x"*1000000)'
docker inspect ce-lab08-mem --format '{{json .State}}'
docker rm ce-lab08-mem
```

Here the equal memory and memory-swap values allow no container swap. Check **OOMKilled**, exit status and kernel logs together. A process can also fail allocation without being selected as the OOM victim.

3. Bound process creation without a fork bomb

```
docker run --name ce-lab08-pids --pids-limit=20 --memory=128m python:3.12-slim python -c '
import subprocess
children=[]
try:
    for i in range(40): children.append(subprocess.Popen(["sleep","15"]))
except OSError as e:
    print(type(e).__name__,str(e),"children",len(children),flush=True)
finally:
    for p in children: p.terminate()
    for p in children: p.wait()'
docker rm ce-lab08-pids
```

Expected evidence: process creation fails before 40 children; the limit also includes the container's existing tasks.

4. Show shared-storage exhaustion in a bounded filesystem

Use a 32 MiB **tmpfs**, not the VM root disk:

```
mkdir -p ~/labs/lab08/shared
sudo mount -t tmpfs -o size=32m tmpfs "$HOME/labs/lab08/shared"
docker run --rm -v "$HOME/labs/lab08/shared:/data" ubuntu:24.04 \
  sh -c 'dd if=/dev/zero of=/data/full bs=1M count=40; df -h /data'
docker run --rm -v "$HOME/labs/lab08/shared:/data" ubuntu:24.04 \
```

```
sh -c 'dd if=/dev/zero of=/data/probe bs=1M count=1'
sudo rm -f ~/labs/lab08/shared/full ~/labs/lab08/shared/probe
docker run --rm -v "$HOME/labs/lab08/shared:/data" ubuntu:24.04 \
  sh -c 'dd if=/dev/zero of=/data/probe bs=1M count=1'
sudo rm -f ~/labs/lab08/shared/probe
sudo umount ~/labs/lab08/shared
```

Why: two containers share one finite filesystem. This demonstrates shared-capacity failure, but not block-disk latency or Docker writable-layer quota behaviour.

Cleanup / recovery: remove only the named `ce-lab08-*` containers above; unmount this exact tmpfs after its users exit. Do not run `docker volume prune`.

Done when: the journal has one observed limit per resource and a recovered storage probe.

If it differs: verify limits in `inspect/cgroup` files and that both storage probes used the same mount. Image pulls happen outside that tmpfs and need free VM disk space.

- Optional depth — continue after the core works

Optional depth: pin two busy containers to one CPU and compare `--cpu-shares=512` with `2048`; v2 weight conversion need not give an exact 20/80 split. Move them to different CPUs to show that weights are not caps. Compare `--memory-swap=128m` only when host swap exists. Compare private PID namespaces versus `--pid=host` using `kill -0`, not `kill -9` with a host PID that could name a different namespace-local process. UID permissions and AppArmor can affect capability experiments. Storage-driver quotas are backend-specific; inspect Docker storage documentation before using `--storage-opt size`.

Lab 9 — Inject delay from outside the target container



Theory before you start

- **Concept:** A network namespace contains interfaces, routes and packet-handling configuration. Containers sharing that namespace share those network settings even if they have separate filesystems. This lets a helper supply diagnostic tools without modifying the application's image. It also means a helper's networking changes can directly affect the target, so the chosen namespace defines the actual scope.
- **Mechanism:** The host can enter MySQL's network namespace, or a helper container can join it and use `NET_ADMIN` permission to configure tc. Both methods install a netem qdisc on the same target interface, delaying database egress. Pumba automates related targeting and cleanup. The qdisc remains attached to the interface after the helper exits; deleting the helper alone does not restore traffic.
- **Read the result:** Verify the target interface and delay counters, then compare application latency. Removing the helper does not undo the networking change: recovery requires deleting the qdisc and rechecking the baseline.

Study-guide basis: Ch. 5 §§5.8–5.11.

Goal: Understand the shared-network-namespace technique before adding an automation tool.

Before you start: Lab 5 running; Lab 6 qdisc removed.

Time: 25–40 min (estimate; downloads excluded).

Core experiment

1. Resolve MySQL and measure WordPress

```
cd ~/labs/lab05
DB=$(docker compose ps -q db)
PID=$(docker inspect -f '{{.State.Pid}}' "$DB")
dbnet() { [ "$PID" -gt 1 ] || return 1; sudo nsenter --target "$PID" --net
-- "$@"; }
dbnet tc qdisc show dev eth0
source ~/labs/common.sh
measure http://localhost:8080/ 10
```

2. Add 25 ms on database egress

Only proceed if the interface has no custom root qdisc:

```
dbnet tc qdisc add dev eth0 root netem delay 25ms
dbnet tc -s qdisc show dev eth0
measure http://localhost:8080/ 10
dbnet tc qdisc del dev eth0 root
measure http://localhost:8080/ 10
```

Prediction: the page can gain more than 25 ms because it waits on several database responses. This delays all MySQL-container egress, unlike Lab 6's port filter.

3. Repeat using a helper container

Build the helper locally so its entrypoint is explicit:

```
mkdir -p ~/labs/lab09 && cd ~/labs/lab09
printf 'FROM ubuntu:24.04\nRUN apt-get update && apt-get install -y\niproute2 && rm -rf /var/lib/apt/lists/*\nENTRYPOINT ["tc"]\n' > Dockerfile
docker build -t ce-tc:lab .
docker run --rm --network "container:$DB" --cap-add NET_ADMIN ce-tc:lab
qdisc add dev eth0 root netem delay 25ms
docker run --rm --network "container:$DB" --cap-add NET_ADMIN ce-tc:lab -s
qdisc show dev eth0
measure http://localhost:8080/ 10
```

```
docker run --rm --network "container:$DB" --cap-add NET_ADMIN ce-tc:lab
qdisc del dev eth0 root
```

Why: `tc` changes the target network namespace even though its executable lives in another container. MySQL needs no debugging tools installed.

Cleanup / recovery: delete the qdisc with either method and remeasure. `--rm` removes the helper container; it does **not** remove the qdisc it installed.

Done when: the host method and helper method show the same mechanism and both recover.

If it differs: check the helper image entrypoint, `NET_ADMIN`, the target's current PID and qdisc counters.

- Optional depth — continue after the core works

Optional depth — Pumba: use a release from the Pumba project, record its version and read `pumba netem --help`. Use the exact container name from `docker inspect -f '{{.Name}}' "$DB"` rather than a prefix regex; verify targets before acting. Compare a zero-delay control and 25 ms with a 30-second `--duration`. Confirm qdisc removal afterward; tool failure can leave state behind.

Optional depth — crash versus hang: `docker pause "$DB"`, probe with a 3-second curl timeout, then `docker unpause "$DB"`; separately use `docker kill "$DB"`, then explicitly `docker start "$DB"` and wait for database readiness. The Compose database has no restart policy, so do not promise automatic restart. Never combine pause and kill in one comparison.

Lab 10 — Inject one syscall error into one process



Theory before you start

- **Concept:** A system call is the boundary where an application asks the kernel to perform work, such as writing bytes or closing a file descriptor. Library functions often invoke syscalls underneath. A syscall can fail even when the application logic is otherwise correct, so reliable software must handle error returns without silently losing work or unnecessarily terminating service.
- **Mechanism:** strace uses tracing facilities to observe calls and can substitute an error such as EIO, meaning an input/output error. In the lab's error-injection mode, the targeted operation is skipped and an artificial failure is returned. This exercises application error handling, but may leave different resource state from a real failure. Tracing also adds overhead, which needs its own control when comparing performance.
- **Read the result:** Confirm an injected call occurred, then inspect response completeness, logs and exit status. For performance comparisons, separate tracer overhead from the injected fault using a traced, non-injecting control.

Study-guide basis: Ch. 6 §§6.2–6.4.

Goal: Distinguish observation overhead, failure injection and application behaviour.

Before you start: Lab 0; stop Lab 5 with docker compose down so port 8080 is free.

Time: 35–50 min (estimate; downloads excluded).

Core experiment

1. Build and run the book example

```
cd ~/labs/book/examples/who-you-gonna-call/src
cc -O0 -o legacy_server $(find . -name '*.c')
./legacy_server > ~/labs/lab10-server.log 2>&1 & server=$!
curl -fsS --max-time 3 http://127.0.0.1:8080/ | head -3
```

Checkpoint: the server starts and answers. Preserve `$server` in this terminal; do not use a global process-name kill.

The direct build avoids the historical Makefile's extra copy into the home directory. It compiles the checked-out C sources rather than assuming a prebuilt server exists.

2. Observe before changing behaviour

In terminal 2, get the PID from the server log or terminal 1, then run `sudo strace -p PID -e trace=write,close,fsync -o ~/labs/lab10-trace.txt`. In terminal 1, request a page. Ctrl-C in terminal 2 detaches; inspect the trace.

Prediction: the trace reveals many writes and any failing syscalls. A failing `fsync` on a socket is not automatically the cause of an application outage.

3. Inject a close error

In terminal 2: `sudo strace -p PID -e trace=close -e inject=close:error=EIO`. Request one page in terminal 1, then `wait "$server"; echo "server exit=$?"` if it exited. Detach the tracer if still running.

Expected evidence: the injected syscall is annotated as injected in strace. The server may terminate on its error-handling path; use the server log and exit status to establish that outcome.

4. Recover and compare

Restart `./legacy_server` and save its new PID. Confirm the baseline URL works. Repeat with `write:error=EIO:when=1+2` only after ending the previous tracer. Measure status, body and throughput; successful status alone does not prove a complete response.

Cleanup / recovery: Ctrl-C the attached tracer, then `kill "$server"; wait "$server"` in the terminal that started that exact process. A fresh server without a tracer is the recovery check.

Done when: you can distinguish syscall failure, application outcome and tracer overhead.

If it differs: confirm the PID, attachment permission, syscall names in the baseline trace and whether the process is still alive. On Linux an EINTR return from close has subtle semantics; an injector that skips

close can create a different descriptor state from a real kernel failure.

- Optional depth — continue after the core works

Optional depth — observer cost: compare `time dd if=/dev/zero of=/dev/null bs=1 count=100000` with the same command under `strace -o /dev/null -e trace=accept`, then `strace -f --seccomp-bpf -o /dev/null -e trace=accept`. Measure ratios on your machine; do not reuse the book's 100× number. BCC `syscount-bpfcc` counts events with different information and overhead.

Optional depth — four controls: test `error=EIO`, `retval=0` on `fsync`, `signal=SIGPIPE` and `delay_enter=100ms` separately. A fake successful return can hide a bug without performing the operation; signal behaviour depends on handlers. A delay applies per matching syscall, not necessarily once per HTTP request. Read your installed `man strace` injection syntax and `man 2 write` / `man 2 close` before selecting errno.

Lab 11 — Use capabilities and seccomp as precise controls



Theory before you start

- **Concept:** Linux capabilities divide privileged authority into named permissions, such as permission to change a process's filesystem root. Seccomp instead filters the system calls a process attempts. An operation may therefore fail because authorization is missing or because a filter rejects the call. Neither control grants permission denied by another layer; several checks can apply to the same operation.
- **Mechanism:** Dropping `SYS_CHROOT` removes the permission needed by the demonstrated chroot operation. The separate C program installs a seccomp filter after startup and makes one selected syscall return an error number. The filter applies to that process and is inherited by its descendants, without changing the whole host. Installing it after startup avoids confusing loader failures with the intended syscall test.
- **Read the result:** Attribute the failure to the control you changed. The teaching filter proves a syscall outcome can be constrained without editing the application; it does not establish a complete security policy or zero overhead.

Study-guide basis: Ch. 5 §5.8.1; Ch. 6 §6.5.

Goal: Make one operation fail without changing its application source.

Before you start: Lab 0; Lab 10 is optional background.

Time: 25–40 min (estimate; downloads excluded).

Core experiment

1. Remove one capability

```
docker run --rm ubuntu:24.04 chroot / true
docker run --rm --cap-drop SYS_CHROOT ubuntu:24.04 chroot / true
docker run --rm --cap-drop ALL ubuntu:24.04 sh -c 'grep CapEff
/proc/self/status'
```

Prediction: removing `SYS_CHROOT` prevents chroot in this default setup. Read the error; do not assume every permission failure comes from the same layer.

2. Apply a small libseccomp filter after startup

This avoids downloading an unpinned default profile or accidentally breaking the dynamic loader. Save as `~/labs/lab11/filter.c`:

```
#include <seccomp.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <errno.h>
#include <stdio.h>
int main(void) {
    scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_ALLOW);
    if (!ctx) return 1;
    if (seccomp_rule_add(ctx, SCMP_ACT_ERRNO(EACCES), SCMP_SYS(getpid), 0)
    < 0)
        return 2;
    if (seccomp_load(ctx) < 0) return 3;
    seccomp_release(ctx);
    errno = 0;
    long r = syscall(SYS_getpid);
    printf("getpid result=%ld errno=%d\n", r, errno);
    return (r == -1 && errno == EACCES) ? 0 : 4;
}
```

```
mkdir -p ~/labs/lab11
cd ~/labs/lab11
cc -Wall -Wextra filter.c -lseccomp -o filter
./filter
```

Checkpoint: `result=-1 errno=13`. The filter acts only on this process and descendants. The program deliberately uses `syscall` to avoid library-wrapper assumptions.

3. Explain the result

The program did not lose general root privileges. The kernel filter replaced one syscall's outcome with an `errno`. This is useful for testing a branch, but an allow-all-except-one teaching filter is not a production hardening policy. Seccomp has overhead; "zero overhead" was too strong. Docker seccomp documentation.

Cleanup / recovery: the process exit removes its filter; no host-wide setting was changed. Remove `filter` if no longer needed.

Done when: you distinguish capability authorization from syscall filtering.

If it differs: compile errors → check `libseccomp-dev`; load errors → inspect the enclosing VM/container security policy and return code.

- Optional depth — continue after the core works

Optional depth: compare a non-root port-80 bind with `net.ipv4.ip_unprivileged_port_start=0` versus `1024`; the `sysctl` can permit binds without `NET_BIND_SERVICE`. Decode `CapEff` with `capsh --decode=HEX`, and measure the actual default capability set. For a Docker JSON filter, start from the default profile matching the engine release, change one syscall and validate JSON with `jq`; keep the original profile. A filter active from process creation can break the loader before the intended request path. The book's deny-by-default C filter also interacts with libc's `exit_group` and fallback `exit`; deleting one allow rule does not justify a universal claim about clean termination.

Lab 12 — Inject a JVM failure and detect silent success



Theory before you start

- **Concept:** Java source becomes bytecode stored in class files, which a compatible JVM loads and executes. Instrumentation changes that executable representation without editing the original method's source. A Java agent can therefore inject a failure at a chosen method boundary. The experiment asks whether the application reports incomplete work correctly, rather than merely whether it survives an exception.
- **Mechanism:** The `-javaagent` option loads an agent that uses ASM, a bytecode-editing library, to transform the selected class. Matching the class, method name and descriptor limits injection to the intended signature. The inserted call throws before normal output; the application catches the exception and continues. Because catching an exception does not automatically make the process exit nonzero, output completeness must be checked independently of exit status.
- **Read the result:** Check three independent signals: the injection ran, expected output exists, and the process exit status. Successful termination is not proof of successful work; missing output can reveal failure hidden by exception handling.

Study-guide basis: Ch. 7 §7.2.

Goal: Understand bytecode instrumentation using a small application you can build directly.

Before you start: Lab 0 with JDK 17; no FizzBuzzEnterpriseEdition build required.

Time: 45–70 min (estimate; downloads excluded).

Core experiment

1. Build a tiny batch application

```
mkdir -p ~/labs/lab12 && cd ~/labs/lab12
cat > Demo.java <<'JAVA'
import java.io.IOException;
public class Demo {
    public static void output(int i) throws IOException {
        System.out.println(i); }
    public static void boom() throws IOException { throw new
        IOException("CHAOS"); }
    public static void main(String[] args) {
        for (int i=1; i<=10; i++) {
            try { output(i); } catch (IOException e) { System.err.println(e); }
        }
    }
}
JAVA
javac --release 17 Demo.java
java Demo > baseline.txt
echo "exit=$?"; wc -l baseline.txt
javap -c Demo
```

Checkpoint: exit 0 and ten output lines. `throws` identifies a possible failure surface but does not prove that the method throws at runtime.

2. Build a narrowly targeted ASM agent

```
curl -fL -o asm-9.7.jar
https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.7/asm-9.7.jar
cat > Agent.java <<'JAVA'
import java.lang.instrument.*;
import java.security.ProtectionDomain;
import org.objectweb.asm.*;
public class Agent {
    public static void premain(String args, Instrumentation inst) {
        inst.addTransformer(new ClassFileTransformer() {
            public byte[] transform(ClassLoader loader, String name, Class<?>
                cls,
                ProtectionDomain domain, byte[] bytes) {
                if (!"Demo".equals(name)) return null;
                ClassWriter writer = new ClassWriter(ClassWriter.COMPUTE_MAXS);
                ClassVisitor visitor = new ClassVisitor(Opcodes.ASM9, writer) {
                    public MethodVisitor visitMethod(int access, String method,
                        String desc,
                        String signature, String[] exceptions) {
                            MethodVisitor base =
                                super.visitMethod(access, method, desc, signature, exceptions);
                            if (!method.equals("output") || !desc.equals("(I)V")) return
                                base;
                            return new MethodVisitor(Opcodes.ASM9, base) {
```

```
        public void visitCode() {
            super.visitCode();
            visitMethodInsn(OpCodes.INVOKESTATIC,"Demo","boom",
                ()V",false);
        }
    };
}
};
new ClassReader(bytes).accept(visitor,0);
return writer.toByteArray();
}
});
}
```

JAVA

```
javac --release 17 -cp asm-9.7.jar Agent.java
printf 'Premain-Class: Agent\nClass-Path: asm-9.7.jar\n\n' > manifest.mf
jar cfm agent.jar manifest.mf Agent*.class
```

This instruments exactly class `Demo`, method `output`, descriptor `(I)V`. Keep the ASM jar beside the agent jar. Class name alone is not enough to constrain an overloaded method.

3. Predict and run

```
java -javaagent:agent.jar Demo > chaos.txt 2> chaos-errors.txt
echo "exit=$?"; wc -l chaos.txt
cat chaos-errors.txt
```

Expected evidence: ten caught exceptions, no normal output, yet exit 0. The process-level success signal missed the work failure.

4. Verify the result as a user would

```
java Demo > recovered.txt
test "$((wc -l < recovered.txt))" -eq 10 && echo 'output check passed'
```

Cleanup / recovery: omit `-javaagent`. Each Java process terminates; the agent is not installed system-wide.

Done when: the journal separates exit status, output completeness and the injection target.

If it differs: inspect `chaos-errors.txt`, classpath and method descriptor; a failed agent startup is not the intended caught-exception experiment. Compile both classes with `--release 17`: a newer JDK's default class format can exceed ASM 9.7's supported format and leave the class untransformed. Check `javap -v Demo | grep 'major version'` for version 61.

Why: a static `()V` call consumes and returns no operand-stack values. It does not require an unconditional `maxStack += 1`. More complex transformations may require recomputing frames as well as maximum stack; this straight-line insertion introduces no new branch.

- Optional depth — continue after the core works

Optional depth — Byteman partial failure: download the pinned Byteman 4.0.23 distribution, unpack in this directory, and save `half.btm`:

```
RULE fail alternate outputs
CLASS Demo
METHOD output(int)
AT ENTRY
IF incrementCounter("outputs") % 2 == 0
DO throw new java.io.IOException("CHAOS")
ENDRULE
```

Run `java -javaagent:byteman-download-4.0.23/lib/byteman.jar=script:half.btm Demo` without the ASM agent. Count the successful lines. For dynamic attach, make a separate long-running version with a one-second pause between iterations; start it with `java Demo & jvm=$!`, use that exact PID with `bminstall.sh`, submit with `bmsubmit.sh -l half.btm`, then unload with `bmsubmit.sh -u half.btm`. Check the Byteman guide for listener/JDK attach settings. Stop the captured JVM PID afterward.

Optional depth — original example: the book's `examples/jvm/Makefile` builds `FizzBuzzEnterpriseEdition` and downloads historical tools; the application jars are not supplied in the source tree. Treat this as a separate historical build exercise. Do not assume OpenJDK 8 is available through Ubuntu 24.04's default apt repositories. Byte-Monkey and Chaos Monkey for Spring Boot illustrate broader injection surfaces; inspect their supported runtime versions before adapting them.

Lab 13 — Expose a race with application-level latency



Theory before you start

- **Concept:** A read-modify-write sequence spans several operations: read the old value, calculate a replacement, then save it. Two requests can both read the same value and each write a different replacement, losing one update. Individually atomic GET and SET commands do not make the whole sequence atomic. Concurrency exposes this gap even if every request returns successfully.
- **Mechanism:** The Redis wrapper delays GET and SET, increasing the time in which requests can overlap. The fix replaces client-side rewriting with list operations grouped in MULTI/EXEC, preventing other clients' commands from interleaving within that transaction. Plain pipelining only batches requests and does not provide that guarantee. Because the fix changes the client interface, the latency injection must move to the transaction boundary too.

- **Read the result:** Check stored data as well as response latency. After the fix, recheck injection coverage: the original GET/SET wrapper does not implement the new transaction interface.

Study-guide basis: Ch. 8 §§8.2–8.4. Redis transactions

Goal: Compare wrapper overhead, graceful degradation and lost updates.

Before you start: Lab 0; Redis working; port 5000 free.

Time: 45–60 min (estimate; downloads excluded).

Core experiment

1. Prepare the book app in an isolated folder

```
sudo systemctl start redis-server
cp -r ~/labs/book/examples/app ~/labs/lab13
cd ~/labs/lab13
python3 -m venv .venv
.venv/bin/pip install 'Flask==3.0.3' 'redis==5.0.8'
cp app.py app-original.py
sed -i 's/^#import chaos$/import chaos/; s/^#CACHE_CLIENT =/CACHE_CLIENT =/' app.py
env -u CHAOS -u CHAOS_DELAY_SECONDS .venv/bin/flask --app app run --no-reload > server.log 2>&1 & appid=$!
```

Checkpoint: `curl -fsS --max-time 3 -b sessionID=ce-lab13 -d query=apple http://127.0.0.1:5000/search` returns a page with interests. Only these lab keys are used; do not flush the Redis database.

2. Compare disabled, zero-delay and 0.1-second delay

```
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{time_total}\n' \
  -b sessionID=ce-lab13 -d query=apple http://127.0.0.1:5000/search
kill "$appid"; wait "$appid" 2>/dev/null || true
CHAOS=1 CHAOS_DELAY_SECONDS=0 .venv/bin/flask --app app run --no-reload >
server.log 2>&1 & appid=$!
```

Wait until the same curl probe succeeds; repeat it five times. Stop that exact PID and restart with `CHAOS_DELAY_SECONDS=0.1`; probe five times again. Do not use `CHAOS=false`: the book checks whether the string exists, so the nonempty string `false` enables it.

Prediction: `/search` performs GET then SET, so latency increases by roughly two injected delays. The zero-delay case isolates wrapper overhead more closely than a nonzero control.

3. Send six concurrent updates to one key

With 0.1-second delay active:

```

for run in $(seq 1 5); do
  redis-cli del ce-lab13-race >/dev/null
  requests=()
  for q in a b c d e f; do
    curl -fsS --max-time 5 -o /dev/null -b sessionID=ce-lab13-race \
      -d "query=$q" http://127.0.0.1:5000/search & requests+=("$!")
  done
  for pid in "${requests[@]}; do wait "$pid"; done
  redis-cli get ce-lab13-race
done

```

Wait only for those requests. A bare `wait` would also wait for the Flask server in this shell and appear to hang. Repeat without CHAOS. The final list should reflect three distinct recent queries; fewer entries can reveal overlapping read-modify-write operations. Race occurrence is probabilistic.

4. Review a real fix and its injection coverage

The book wrapper has only `get` and `set`: calling `pipeline()` raises `AttributeError`, rather than silently forwarding. Stop the app, replace `store_interests` with this function and add `import os, time` near the top of `app.py`. This teaching variant unwraps the client explicitly and moves the latency gate to the transaction boundary:

```

def store_interests(session, query):
    client = CACHE_CLIENT.client if isinstance(CACHE_CLIENT,
chaos.ChaosClient) else CACHE_CLIENT
    key = 'ce-lab13-list:' + session
    with client.pipeline(transaction=True) as pipe:
        if query:
            pipe.lrem(key, 0, query)
            pipe.rpush(key, query)
        pipe.ltrim(key, -3, -1)
        pipe.lrange(key, 0, -1)
        if os.environ.get('CHAOS'):
            time.sleep(float(os.environ.get('CHAOS_DELAY_SECONDS', '0')))
        values = pipe.execute()[-1]
    return [value.decode() for value in values]

```

This uses a **new list key**, avoiding `WRONGTYPE` on existing JSON-string keys. Restart with the same latency settings. In step 3 replace the delete with `redis-cli del ce-lab13-list:ce-lab13-race` and the read with `redis-cli lrange ce-lab13-list:ce-lab13-race 0 -1`. After six successful distinct concurrent requests, three entries should remain; order reflects transaction execution order, not client launch order. The three list-update commands run atomically inside `MULTI/EXEC`. Redis transaction semantics.

The injection now delays the whole transaction once, so the old “two delays per request” expectation no longer applies. Compare zero versus 0.1 seconds again to verify coverage. This fixes lost updates for this narrow example; it is not a complete production caching API or a fix for every failure mode.

Cleanup / recovery: `kill "$appid"; wait "$appid" 2>/dev/null || true; redis-cli del ce-lab13 ce-lab13-race ce-lab13-list:ce-lab13 ce-lab13-list:ce-lab13-race; unset CHAOS CHAOS_DELAY_SECONDS`. Keep your source changes for comparison.

Done when: you have latency comparisons and evidence about lost updates, with the injector's interface limitation stated correctly.

If it differs: inspect `server.log`, ensure no reloader/second server owns port 5000 and verify the wrapper is actually enabled.

- Optional depth — continue after the core works

Optional depth: restore `app-original.py` to `app.py`, uncomment `import chaos2` and the decorator immediately above `get_interests`, restart with `CHAOS=1`, and compare alternating recommendation outcomes. HTTP 200 can still represent missing recommendations. Add a checkout-style endpoint and specify whether it should fail or degrade when Redis is unavailable; test that decision rather than copying the recommendation endpoint's policy.

Lab 14 — Test browser delay, failure and stale data



Theory before you start

- **Concept:** Browser resilience covers waiting, errors and the truthfulness of displayed data. A request can fail at transport level, return an HTTP error, or succeed with content the application cannot use. These require deliberate UI handling. Leaving the previous result visible without marking it as stale can make a failed refresh look successful, even though nothing current was retrieved.
- **Mechanism:** The temporary fetch wrapper either waits before calling the original function or rejects the promise without sending a request. This separates application waiting from network timing. A real HTTP 404 or 500 normally resolves fetch, so `response.ok` must be checked; transport failure can reject it. Measure click-to-result time, then compare synthetic failure with browser request blocking to understand what each injection actually models.
- **Read the result:** Measure click-to-result time as well as network timing. On failure, the UI should identify the problem and avoid presenting an old value as fresh. Refreshing removes this page's temporary wrapper.

Study-guide basis: Ch. 9 §§9.2–9.4. Fetch error handling

Goal: Observe user-visible behaviour with a small local page.

Before you start: Lab 0; a browser on the laptop; port 8081 free.

Time: 25–40 min (estimate; downloads excluded).

Core experiment

1. Create a local fixture

```
mkdir -p ~/labs/lab14 && cd ~/labs/lab14
printf '{"value":"fresh data"}\n' > data.json
cat > index.html <<'HTML'
<!doctype html><meta charset="utf-8"><title>Browser chaos lab</title>
<button id="load">Load data</button><pre id="result">No data yet</pre>
<script>
document.querySelector('#load').onclick = async () => {
  const box = document.querySelector('#result');
  try {
    const r = await fetch('/data.json?t='+Date.now());
    if (!r.ok) throw new Error('HTTP '+r.status);
    box.textContent = (await r.json()).value;
  } catch (e) { box.textContent = 'Unavailable: '+e.message; }
};
</script>
HTML
python3 -m http.server 8081 --bind 127.0.0.1
```

On the laptop: `ssh -L 8081:127.0.0.1:8081 VM_USER@VM_ADDRESS`; open `http://localhost:8081`. Keep the server in its foreground terminal.

2. Establish the baseline

Open DevTools → Network. Click Load data three times. Expect one data request per click and visible fresh data. Record request duration and HTTP status.

3. Inject delay and alternate failure

Paste into **this page's** browser console once:

```
window.ceOriginalFetch = window.fetch;
window.ceCount = 0;
window.fetch = async function (...args) {
  window.ceCount++;
  if (window.ceCount % 2 === 0) throw new TypeError('Injected network failure');
  await new Promise(resolve => setTimeout(resolve, 1000));
  return window.ceOriginalFetch.apply(this, args);
};
```

Prediction: alternate clicks show a delayed success or an explicit error. The delay before calling native fetch can be missing from the network request's own duration; measure click-to-result too. Synthetic rejection may create no network request at all.

4. Compare zero-code failure and recover

Refresh the page to restore the original function. Use DevTools request blocking for `data.json`, or Offline mode, then click again. Restore normal networking and reload.

Cleanup / recovery: page refresh removes this monkey patch; disabling request blocking/Offline is a separate action. Ctrl-C the server after the recovery click succeeds.

Done when: you distinguish transport timing, UI timing, HTTP errors and network rejection.

If it differs: refresh before repasting; inspect the console for syntax errors and check that DevTools targets the local fixture.

Why: fetch rejects on transport failures but normally resolves for HTTP 4xx/5xx, so the app checks `r.ok`. An error handler must not assume a missing response contains JSON.

- Optional depth — continue after the core works

Optional depth — stale data: change the catch branch to log only, reload and repeat. The previous successful value stays visible, illustrating why an apparently healthy UI can mislead users. Restore the explicit error afterward.

Optional depth — XHR: study `~/labs/book/examples/js/XMLHttpRequest-2.js` for a delayed `send` that captures arguments and `this`. Synthetic error-event dispatch does not reproduce the full browser transport lifecycle; compare it against request blocking. The book's pgweb 0.11.6 reproduction is a historical case, not a promise about current pgweb. Tampermonkey can install a patch at document-start when code caches native references; use an exact `localhost` match and disable the script after the exercise.

Lab 15 — Delete one Kubernetes pod and measure recovery



Theory before you start

- **Concept:** A Pod groups containers that run together; a Deployment manages the desired number through a ReplicaSet. A Service selects backends and provides stable access while individual Pods change. Ready expresses whether a Pod is eligible to serve according to configured checks; Running only describes its lifecycle phase. RBAC separately controls which API actions the application's identity may perform.
- **Mechanism:** Deleting a managed Pod causes the replica controller to create a new object with a new identity. The scheduler chooses a node, kubelet starts its containers, and readiness and endpoint information update asynchronously. Remaining replicas may serve during that gap if they have enough capacity. The replacement is not the old Pod restarting, and a TCP readiness check establishes a listening socket rather than complete application health.
- **Read the result:** Distinguish Running from Ready, and replacement from user-visible recovery. Track pod identity, ready endpoints and HTTP samples. This lab's TCP readiness proves a listening port, not complete application correctness.

Study-guide basis: Ch. 10 §§10.4–10.4.2.

Goal: Separate desired replicas, Ready endpoints and sampled HTTP availability.

Before you start: Lab 0; rootful Docker in the VM; 8–12 GB free RAM recommended.

Time: 45–70 min (estimate; downloads excluded).

Core experiment

1. Install a recorded kind/kubectrl pair

Use the kind release page to select a kind release and a supported node image. The example uses kind v0.33.0; record the node image it creates. Install the kubectrl minor version matching the resulting API server instead of blindly using “latest”. Architecture-aware kind download:

```
case "$(uname -m)" in x86_64) arch=amd64;; aarch64) arch=arm64;; *) echo
unsupported; exit 1;; esac
curl -fL -o /tmp/ce-kind "https://kind.sigs.k8s.io/dl/v0.33.0/kind-
linux-$arch"
sudo install -m 0755 /tmp/ce-kind /usr/local/bin/kind
mkdir -p ~/labs/lab15
```

Save ~/labs/lab15/kind.yaml:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: chaos
nodes:
- role: control-plane
- role: worker
  labels: {topology.kubernetes.io/zone: west-1}
- role: worker
  labels: {topology.kubernetes.io/zone: west-2}
- role: worker
  labels: {topology.kubernetes.io/zone: east-1}
```

```
kind create cluster --config ~/labs/lab15/kind.yaml --kubeconfig
~/labs/chaos.kubeconfig --wait 180s
docker exec chaos-control-plane kubeadm version -o short
docker inspect chaos-control-plane --format '{{.Config.Image}}' | tee
~/labs/lab15/node-image.txt
```

Take the exact version printed by kubeadm (for example, v1.MINOR.PATCH), put it in KUBE_VERSION, and download the matching kubectrl with the same \$arch:

```
KUBE_VERSION=$(docker exec chaos-control-plane kubeadm version -o short)
curl -fL -o /tmp/ce-kubectrl
"https://dl.k8s.io/release/$KUBE_VERSION/bin/linux/$arch/kubectrl"
```

```
curl -fL -o /tmp/ce-kubectl.sha256
"https://dl.k8s.io/release/$KUBE_VERSION/bin/linux/$arch/kubectl.sha256"
echo "$(cat /tmp/ce-kubectl.sha256) /tmp/ce-kubectl" | sha256sum --check
sudo install -m 0755 /tmp/ce-kubectl /usr/local/bin/kubectl
```

2. Use a dedicated cluster helper in every terminal

Append to `~/labs/common.sh`, then source it:

```
k() { kubectl --kubeconfig="$HOME/labs/chaos.kubeconfig" --context=kind-
chaos --namespace=chaos-labs --request-timeout=10s "$@"; }
```

```
source ~/labs/common.sh
k create namespace chaos-labs
k get nodes
```

All later `k` commands use this exact lab cluster and namespace, independent of your default context.

3. Deploy complete Goldpinger RBAC and workload

Save `~/labs/lab15/goldpinger.yaml`:

```
apiVersion: v1
kind: ServiceAccount
metadata: {name: goldpinger}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: goldpinger}
rules:
- apiGroups: ['']
  resources: [pods]
  verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: goldpinger}
subjects: [{kind: ServiceAccount, name: goldpinger, namespace: chaos-labs}]
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: Role, name:
goldpinger}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: goldpinger}
spec:
  replicas: 3
  selector: {matchLabels: {app: goldpinger}}
```

```

template:
  metadata: {labels: {app: goldpinger}}
  spec:
    serviceAccountName: goldpinger
    containers:
      - name: goldpinger
        image: bloomberg/goldpinger:v3.11.3
        env:
          - {name: HOST, value: '0.0.0.0'}
          - {name: PORT, value: '8080'}
          - {name: CLIENT_PORT_OVERRIDE, value: '8080'}
          - {name: PING_TIMEOUT, value: '300ms'}
          - {name: REFRESH_INTERVAL, value: '2'}
          - {name: NAMESPACE, value: chaos-labs}
          - name: HOSTNAME
            valueFrom: {fieldRef: {fieldPath: metadata.name}}
          - name: POD_IP
            valueFrom: {fieldRef: {fieldPath: status.podIP}}
        ports: [{name: http, containerPort: 8080}]
        readinessProbe:
          tcpSocket: {port: http}
          periodSeconds: 2
        resources:
          requests: {cpu: 20m, memory: 64Mi}
          limits: {memory: 256Mi}
---
apiVersion: v1
kind: Service
metadata: {name: goldpinger}
spec:
  type: NodePort
  selector: {app: goldpinger}
  ports: [{name: http, port: 8080, targetPort: 8080, nodePort: 30080}]

```

```

k apply -f ~/labs/lab15/goldpinger.yaml
k rollout status deployment/goldpinger --request-timeout=0 --timeout=180s
k auth can-i list pods --as=system:serviceaccount:chaos-labs:goldpinger
k auth can-i delete pods --as=system:serviceaccount:chaos-labs:goldpinger
NODE_IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}'
chaos-control-plane)
curl -fsS --max-time 3 "http://$NODE_IP:30080/healthz"
k get endpointslices -l kubernetes.io/service-name=goldpinger -o wide

```

Checkpoint: three Ready pods; list=yes, delete=no; HTTP 200. TCP readiness only proves the listener accepts connections; it deliberately avoids removing all peers when cluster-wide health degrades.

4. Record before deleting one pod

Terminal 2 (source ~/labs/common.sh and resolve NODE_IP there too):

```
for i in $(seq 1 100); do
  code=$(curl -s --max-time 1 -o /dev/null -w '%{http_code}'
"http://$NODE_IP:30080/healthz")
  printf '%s %s\n' "$(date --iso-8601=ns)" "$code"
  sleep 0.2
done | tee ~/labs/lab15/probes.log
```

Terminal 1:

```
k get pods -l app=goldpinger -o wide
victim=$(k get pods -l app=goldpinger -o
jsonpath='{.items[0].metadata.name}')
k delete pod "$victim" --wait=false
k get pods -l app=goldpinger --request-timeout=0 -w
```

Ctrl-C the watch after three replacements/remaining pods are Ready. Read pod Ready conditions, not just **Running**; record recovery time and probe gaps. `grep -vc ' 200$' ~/labs/lab15/probes.log` counts non-200 samples.

Cleanup / recovery: `k rollout status deployment/goldpinger --request-timeout=0 --timeout=180s`. Keep this cluster for 16–18; after them use `kind delete cluster --name chaos`.

Done when: the deleted pod's name differs from its replacement and the recovery is backed by Ready/EndpointSlice and HTTP evidence.

If it differs: inspect pod events, logs, image pulls, Ready condition and RoleBinding. A NodePort is reachable from the Linux Docker host used here; that assumption differs on Docker Desktop.

- Optional depth — continue after the core works

Optional depth: compare graceful delete with force delete, then two of three pods. A force delete removes the API object without confirming process termination. Compare TCP readiness with `/healthz` readiness and consider peer-failure feedback. Create a PDB with `minAvailable: 3`, then compare direct delete with `drain`: PDBs constrain eviction, not direct pod deletion. Uncordon the drained node and delete the PDB afterward. For RBAC failure, delete only this namespace's RoleBinding, observe logs, then reapply the manifest; never delete unrelated cluster-wide bindings.

Lab 16 — Add one deliberately slow replica



Theory before you start

- **Concept:** Health is often reported as a binary result even though performance degrades gradually. A replica may accept connections but respond too late for its callers. A timeout is the caller's

waiting budget, not a statement that the server stopped working. Goldpinger's peer checks let this lab observe how increasing latency crosses that practical boundary.

- **Mechanism:** Toxiproxy is a TCP relay with configurable faults called toxics. Other peers connect to its listener, which forwards to Goldpinger in the same Pod; the upstream toxic here delays data travelling from peers toward that replica. As delay plus normal processing consumes the ping budget, successful checks can become timeouts. The proxy can keep accepting connections throughout, so TCP readiness and peer-level health can disagree.
- **Read the result:** Compare per-peer latency with health as delay increases. A listening port can remain Ready while peer requests time out. Confirm calls traverse the proxy; adding a degraded replica can affect the surrounding system.

Study-guide basis: Ch. 10 §10.4.4.

Goal: Compare a binary health threshold with increasing latency.

Before you start: Lab 15 healthy; source `~/labs/common.sh` in each terminal.

Time: 30–45 min (estimate; downloads excluded).

Core experiment

1. Add one proxy-fronted pod

Save `~/labs/lab15/slow.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: goldpinger-slow
  labels: {app: goldpinger, experiment: lab16}
spec:
  serviceAccountName: goldpinger
  containers:
  - name: goldpinger
    image: bloomberg/goldpinger:v3.11.3
    env:
      - {name: HOST, value: '0.0.0.0'}
      - {name: PORT, value: '9090'}
      - {name: CLIENT_PORT_OVERRIDE, value: '8080'}
      - {name: PING_TIMEOUT, value: '300ms'}
      - {name: REFRESH_INTERVAL, value: '2'}
      - {name: NAMESPACE, value: chaos-labs}
      - name: HOSTNAME
        valueFrom: {fieldRef: {fieldPath: metadata.name}}
      - name: POD_IP
        valueFrom: {fieldRef: {fieldPath: status.podIP}}
    ports: [{containerPort: 9090}]
  - name: toxiproxy
    image: ghcr.io/shopify/toxiproxy:2.12.0
    args: ['-host=0.0.0.0']
    ports: [{containerPort: 8474}, {containerPort: 8080}]
```

```
readinessProbe:
  tcpSocket: {port: 8080}
  periodSeconds: 2
```

```
k apply -f ~/labs/lab15/slow.yaml
k get pod goldpinger-slow --request-timeout=0 -w
```

Wait until both containers are running; **Ready** remains false until the proxy listener exists. Ctrl-C the watch. In a separate terminal: **k port-forward pod/goldpinger-slow 8474:8474**. Wait for "Forwarding from" before the next step.

2. Configure the proxy and check baseline health

```
T=http://127.0.0.1:8474
curl -fsS "$T/version"
curl -fsS -H 'Content-Type: application/json' -X POST "$T/proxies" \
  -d
  '{"name":"slow","listen":"0.0.0.0:8080","upstream":"127.0.0.1:9090","enabled":true}'
k wait --for=condition=Ready pod/goldpinger-slow --request-timeout=0 --
timeout=60s
```

Resolve **NODE_IP** as in Lab 15; fetch **/cluster_health**. Wait several refresh cycles until the new pod is reachable. If that does not happen, inspect pod logs before injecting. Goldpinger's versioned configuration documents the client port and timeout controls.

3. Add one toxic and change its dose

```
curl -fsS -H 'Content-Type: application/json' -X POST
"$T/proxies/slow/toxics" \
  -d
  '{"name":"lat","type":"latency","stream":"upstream","toxicity":1,"attributes":{"latency":50,"jitter":0}}'
for ms in 50 150 250 400; do
  curl -fsS -H 'Content-Type: application/json' -X PATCH
  "$T/proxies/slow/toxics/lat" \
    -d '{"attributes":{"latency":$ms,"jitter":0}}'
  sleep 8
  echo "configured delay=$ms ms"
  curl -sS --max-time 3 "http://$NODE_IP:30080/cluster_health" | jq .
done
```

HTTP error status can be meaningful during an unhealthy cluster, so the health-result read omits **curl -f**; configuration requests keep it. Keep the entire JSON response rather than assuming a field named **OK**. PATCH is supported by the pinned Toxiproxy v2.12.0 API.

Prediction: latency rises before the 300 ms ping budget is exhausted; above it, peer calls fail. Exact transition includes processing and scheduling time. Responses sampled through NodePort may come from different replicas.

4. Remove the fault and verify recovery

```
curl -fsS -X DELETE "$T/proxies/slow/toxics/lat"  
curl -fsS "$T/proxies/slow/toxics"  
sleep 8  
curl -sS --max-time 3 "http://$NODE_IP:30080/cluster_health" | jq .  
k delete pod goldpinger-slow --wait=true
```

Cleanup / recovery: delete the pod and Ctrl-C the port-forward; confirm the original three-pod cluster recovers.

Done when: you record at least one healthy-but-slower case, an unhealthy case or a explained inconclusive result, and recovery.

If it differs: check proxy configuration, JSON request failures, explicit **PING_TIMEOUT**, and which peer each metric describes. Port-forwarding a Service selects a pod; it does not test normal Service balancing.

- Optional depth — continue after the core works

Optional depth: inspect **/metrics** for per-peer latency; identify histogram names and labels before querying. Try **toxicity:0.5**, **timeout** with **timeout:0**, then **reset_peer** separately, deleting the prior toxic each time. These model mixed connections, silence and reset; they are not identical to packet loss or a SYN refusal. Adding a bad replica can still affect users and monitoring, so "added capacity" is a smaller target, not zero risk.

Lab 17 — Automate five startup checks



Theory before you start

- **Concept:** An SLI measures a service outcome; an SLO sets a target for that measurement over a stated scope and window. This lab measures time from workload creation until an HTTP probe succeeds. That includes platform startup work, not just application execution. A repeated check samples variability, but five successful samples cannot establish a long-term availability or latency guarantee.
- **Mechanism:** The loop creates a disposable workload, starts timing, probes its exposed endpoint, records the result and removes its objects before repeating. Scheduling, image retrieval, startup and routing can all affect the duration. A deliberately delayed server acts as a known failing case to validate the checker. Cleanup prevents an old endpoint from producing false success; timestamps expose a checker that has stopped reporting.

- **Read the result:** Five runs demonstrate the checking mechanism, not long-term reliability. Record each duration and outcome; monitor missing results separately from failed results if later automated. Silence from a broken checker is not success.

Study-guide basis: Ch. 11 §§11.1–11.2.

Goal: Turn an experiment into a bounded, observable repeatable check.

Before you start: Lab 15; Lab 16 slow pod removed.

Time: 35–50 min (estimate; downloads excluded).

Core experiment

1. Define one disposable workload

```
mkdir -p ~/labs/lab17 && cd ~/labs/lab17
cat > startup.yaml <<'YAML'
apiVersion: v1
kind: Pod
metadata: {name: startup-check, labels: {app: startup-check}}
spec:
  containers:
  - name: web
    image: python:3.12-slim
    imagePullPolicy: IfNotPresent
    command: [python, -m, http.server, '8080']
    readinessProbe:
      httpGet: {path: /, port: 8080}
      periodSeconds: 1
    restartPolicy: Never
---
apiVersion: v1
kind: Service
metadata: {name: startup-check}
spec:
  type: NodePort
  selector: {app: startup-check}
  ports: [{port: 8080, targetPort: 8080, nodePort: 30090}]
YAML
```

2. Save a finite loop with cleanup

Save as `~/labs/lab17/run.sh`:

```
#!/usr/bin/env bash
set -u
source "$HOME/labs/common.sh"
cd "$HOME/labs/lab17" || exit 1
NODE_IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}')
```

```

chaos-control-plane) || exit 1
cleanup() { k delete -f startup.yaml --ignore-not-found --wait=true --
timeout=30s >/dev/null; }
trap cleanup EXIT
trap 'exit 130' INT TERM
mkdir -p metrics
for run in $(seq 1 5); do
  cleanup || exit 1
  start=$(date +%s)
  ok=0
  k apply -f startup.yaml >/dev/null || exit 1
  while (( $(date +%s)-start < 30 )); do
    if curl -fsS --max-time 1 -o /dev/null "http://$NODE_IP:30090/"; then
ok=1; break; fi
    sleep 0.5
  done
  elapsed=$(( $(date +%s)-start ))
  (( elapsed <= 30 )) || ok=0
  printf 'run=%s success=%s elapsed=%ss\n' "$run" "$ok" "$elapsed"
  printf 'ce_start_success %s\nce_start_seconds
%s\nce_last_run_timestamp_seconds %s\n' \
    "$ok" "$elapsed" "$(date +%s)" > metrics/start.prom.tmp
  mv metrics/start.prom.tmp metrics/start.prom
  k get pod startup-check -o wide
  sleep 2
done

```

3. Predict, then run

`bash run.sh | tee runs.txt`. The proposed lab threshold is **HTTP 200 within approximately 30 seconds**, not a universal platform SLO. Curl and API calls can overrun a coarse wall-clock deadline; record elapsed time and set success to zero if an observed success exceeded your strict acceptance bound.

Checkpoint: five results and no leftover startup pod/service. Inspect events after failures rather than treating slow image pulls as application faults.

4. Prove the check detects failure

Copy `startup.yaml` to `startup-good.yaml`. Change the container command to `[sh, -c, 'sleep 45; exec python -m http.server 8080']`, rerun five cycles, then restore the original file. Each cycle should fail the 30-second test. This controlled startup delay tests the checker without changing node networking.

Cleanup / recovery: the script trap deletes only its manifest objects; manual fallback is `k delete -f ~/labs/lab17/startup.yaml --ignore-not-found`. Restore the good command and confirm a healthy cycle.

Done when: the check distinguishes good and deliberately delayed startup, and cleans up on Ctrl-C.

If it differs: ensure the previous pod/service is gone before timing; a stale endpoint can produce false success. Check image pull, Ready state and NodePort path independently.

- Optional depth — continue after the core works

Optional depth — metrics and dead-man alert: configure a Node Exporter textfile collector to read the absolute `~/labs/lab17/metrics` directory (mount it read-only into the exporter) and scrape it with Prometheus. Validate `/metrics` contains all three `ce_*` gauges before adding:

```
groups:
- name: chaos-lab
  rules:
  - alert: StartupCheckFailed
    expr: ce_start_success == 0 or ce_start_seconds > 30
  - alert: StartupCheckMissing
    expr: (time() - ce_last_run_timestamp_seconds > 120) or
    absent(ce_last_run_timestamp_seconds)
```

The `absent` branch detects missing series, not just old timestamps. Add a separate `up == 0` scrape alert. These gauges retain only the latest run; use per-run logs/counters or a results store for history and error-budget calculation. The finite script intentionally stops after five runs; a stale alert afterward is expected. Prometheus absent semantics.

Optional depth — warm versus cold: `Always` resolves image identity but can reuse cached layers. Use a new disposable kind cluster/node with no image cache for a genuinely cold comparison; `crictl rmi` alone does not prove shared content layers disappeared. Record scheduling, pulling and Ready timestamps. Kubernetes image pull behaviour.

Optional depth — PowerfulSeal: inspect the book's actual `examples/kubernetes/experiment1b.yml`, `experiment2b.yml` and `experiment3.yml`: match→filter→act; add degraded clones; test startup continuously. Copy them into this lab, change namespace to `chaos-labs`, selectors/images explicitly, and validate against the chosen tool schema before running. Do not give a legacy tool your general kubeconfig. This is an optional historical tool exercise; the core loop teaches the continuous-verification pattern without making compatibility assumptions.

Lab 18 — Distinguish kubelet loss from machine loss



Theory before you start

- **Concept:** The kubelet is the node-local agent that reconciles assigned Pods with the container runtime and reports health. Stopping it interrupts supervision and reporting but does not inherently terminate existing containers. Stopping the node removes a much broader part of the execution environment. Kubernetes must infer loss of contact, so failure detection and application recovery are separate events.

- **Mechanism:** Missing heartbeats lead the control plane to mark the node unhealthy or unreachable. NoExecute taints can cause Pods to be evicted after their applicable toleration period; a controller then creates replacements that still need capacity, scheduling and startup. The disconnected node's old processes may continue running. Shortening a toleration changes the eviction wait, not heartbeat detection, and does not guarantee faster end-to-end recovery.
- **Read the result:** Measure detection and workload recovery separately. An API object disappearing does not prove its old process stopped. Recovery time is the combined sequence, not just a single eviction timer.

Study-guide basis: Ch. 11 §11.3; Ch. 12 §12.1.2. Node health and heartbeats

Goal: Measure the detection and eviction timers separately.

Before you start: Lab 15; restore all nodes between experiments.

Time: 40–70 min including waits (estimate; downloads excluded).

Core experiment

1. Put one workload on a known worker

Save `~/labs/lab15/node-web.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: node-web}
spec:
  replicas: 1
  selector: {matchLabels: {app: node-web}}
  template:
    metadata: {labels: {app: node-web}}
    spec:
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - {key: kubernetes.io/hostname, operator: In, values: [chaos-
worker]}
      containers:
        - name: web
          image: nginx:1.27
          readinessProbe: {httpGet: {path: /, port: 80}, periodSeconds: 2}
```

```
k apply -f ~/labs/lab15/node-web.yaml
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
k get pods -l app=node-web -o wide
k get pods -l app=node-web -o jsonpath='{.items[0].spec.tolerations}'
```

Checkpoint: the pod is Ready on `chaos-worker`. If placed elsewhere, use its actual worker in the commands below. This affinity is preferred, so failover can use another worker.

2. Start timeline watches

Terminal 2: `k get nodes --request-timeout=0 -w`. Terminal 3: `k get pods -l app=node-web -o wide --request-timeout=0 -w`. Record wall-clock timestamps alongside observations, including Ready condition changes and deletionTimestamp. End the watches with Ctrl-C.

3. Stop only kubelet

```
date --iso-8601=seconds
docker exec chaos-worker systemctl stop kubelet
docker exec chaos-worker crictl ps --name nginx
```

Wait for node Ready to become Unknown/False and for replacement scheduling. Default not-ready/unreachable NoExecute tolerations are commonly 300 seconds; inspect the pod rather than promise a fixed total recovery time. Stop after eight minutes if no replacement appears and inspect events.

Prediction: existing containers can continue while kubelet is stopped. Detection, toleration expiry, deletion, scheduling, startup and readiness are distinct stages. Kubernetes taints and tolerations.

4. Restore before the next fault

```
docker exec chaos-worker systemctl start kubelet
k wait --for=condition=Ready node/chaos-worker --request-timeout=0 --timeout=180s
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
```

Cleanup / recovery: start the exact kubelet or worker container you stopped; confirm all nodes Ready. Delete only `deployment/node-web` when finished. Keep the cluster only while doing extensions.

Done when: the journal has measured detection and workload recovery times, and distinguishes an API pod object from a process still running on a disconnected node.

If it differs: read effective controller settings, tolerations, events and capacity. A replacement can remain Pending; that is not the same as no eviction.

- Optional depth — continue after the core works

Optional depth — machine death: after recovery, stop the worker with `docker stop chaos-worker`; restore with `docker start chaos-worker`. Compare runtime evidence: stopping kubelet leaves containers running; stopping the node container changes the whole node environment.

Optional depth — shorter toleration: add explicit `not-ready` and `unreachable` NoExecute tolerations of 30 seconds to the Deployment, roll it out and repeat. This changes the eviction delay, not node failure detection.

Optional depth — control-plane partition: inside only one worker's network namespace, add a dedicated commented firewall rule for API-server IP/port 6443, with an exact delete command prepared. A broad INPUT/OUTPUT IP block also affects traffic beyond the API connection. Change `VERSION` on the Deployment, compare runtime processes with API objects, remove the rule and verify convergence. No stateful writers in this exercise.

Optional depth — zones and sandbox: the west-1/west-2/east-1 labels are simulated topology on one physical host, not independent cloud regions. Inspect `nodeTaintsPolicy`, eligible domains and affinity before predicting spread behaviour after losing two workers. `DoNotSchedule` can leave replicas Pending; `ScheduleAnyway` trades distribution for placement. For a selected pod, use `crictl inspectp` to record sandbox identity/IP, stop only its app container and compare after restart. Killing a verified pause PID rebuilds the sandbox; a changed IP is possible, not guaranteed. Check PID > 1 and exact pod identity before any signal. Topology-spread rules.

Lab 19 — Lose etcd quorum and compare control plane with data plane



Theory before you start

- **Concept:** The control plane stores and reconciles desired cluster state; the data plane carries application requests. etcd protects agreement on stored state using Raft consensus: a leader must obtain a voting majority to commit changes. This prevents separated minorities from independently accepting conflicting writes. Losing management availability therefore need not immediately stop an already-established application traffic path.
- **Mechanism:** Three voting members require two for quorum. After one member stops, the remaining majority can commit writes, with leader election if necessary; after two stop, it cannot. Existing processes and installed routing may continue serving HTTP while API writes fail. Probe both paths because some reads can be cached. Restoring the same members tests regained consensus, rather than backup restoration after permanent data loss.
- **Read the result:** Probe application HTTP and a new API write independently. A cached API read can succeed without proving quorum is working. This experiment tests availability during quorum loss, not recovery from permanent data loss.

Study-guide basis: Ch. 12 §12.1.1. etcd quorum

Goal: Observe consensus availability without confusing cached reads with working writes.

Before you start: Lab 15 tooling; delete chaos cluster first if memory is limited.

Time: 60–90 min core; extensions separately (estimate; downloads excluded).

Core experiment

1. Create a separate HA cluster

```
mkdir -p ~/labs/lab19 && cd ~/labs/lab19
cat > kind.yaml <<'YAML'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: ha
nodes:
- role: control-plane
- role: control-plane
- role: control-plane
- role: worker
YAML
kind create cluster --config kind.yaml --kubeconfig ~/labs/ha.kubeconfig --
wait 180s
```

Save these helpers in `~/labs/lab19/helpers.sh` and source them in both terminals:

```
h() { kubectl --kubeconfig="$HOME/labs/ha.kubeconfig" --context=kind-ha --
namespace=default --request-timeout=5s "$@"; }
stop_etcd() { docker exec "$1" mv /etc/kubernetes/manifests/etcd.yaml
/root/ce-etcd.yaml; }
start_etcd() { docker exec "$1" mv /root/ce-etcd.yaml
/etc/kubernetes/manifests/etcd.yaml; }
ec() {
  local node=$1 container; shift
  container=$(docker exec "$node" crictl ps --name etcd -q)
  [ -n "$container" ] || return 1
  docker exec "$node" crictl exec "$container" etcdctl \
    --endpoints=https://127.0.0.1:2379 --dial-timeout=3s --command-
timeout=5s \
    --cacert=/etc/kubernetes/pki/etcd/ca.crt \
    --cert=/etc/kubernetes/pki/etcd/server.crt \
    --key=/etc/kubernetes/pki/etcd/server.key "$@"
}
```

2. Establish HTTP and write baselines

```
source ~/labs/lab19/helpers.sh
h create deployment quorum-web --image=nginx:1.27 --replicas=2
h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
h expose deployment quorum-web --type=NodePort --port=80
```

```
PORT=$(h get svc quorum-web -o jsonpath='{.spec.ports[0].nodePort}')
IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' ha-
worker)
curl -fsS --max-time 3 "http://$IP:$PORT/" >/dev/null
h create configmap quorum-probe --from-literal=state=baseline
ec ha-control-plane endpoint status --cluster -w table
```

Checkpoint: two Ready web pods, successful HTTP, successful ConfigMap write, three healthy etcd members.

3. Stop one etcd member

```
stop_etcd ha-control-plane2
sleep 30
docker exec ha-control-plane2 crictl ps --name etcd
h patch configmap quorum-probe --type merge -p '{"data":{"state":"one-
down"}}'
curl -sS --max-time 3 -o /dev/null -w '%{http_code}\n' "http://$IP:$PORT/"
```

Check the runtime really stopped the static pod; manifest detection is asynchronous. A three-member cluster retains quorum with two healthy members.

4. Stop a second member, probe, then restore

```
stop_etcd ha-control-plane3
sleep 30
h patch configmap quorum-probe --type merge -p '{"data":{"state":"two-
down"}}'
curl -sS --max-time 3 -o /dev/null -w '%{http_code}\n' "http://$IP:$PORT/"
start_etcd ha-control-plane2
start_etcd ha-control-plane3
sleep 30
ec ha-control-plane endpoint health --cluster
h patch configmap quorum-probe --type merge -p '{"data":
{"state":"recovered"}}'
```

Prediction: writes fail or time out without quorum; existing application traffic can continue if its running replicas and routing remain usable. Some GETs can succeed from cache, so a successful `get nodes` alone does not prove consensus works. This lab tests availability, not disaster-recovery restore.

Cleanup / recovery: restore both moved manifests using Docker even when kubectl is unavailable; if state cannot be recovered, `kind delete cluster --name ha`. Backups stay outside the watched manifest directory to avoid duplicate static pods.

Done when: you have independent HTTP and write results for baseline, one-down, two-down and recovery.

If it differs: verify the actual member count, runtime shutdown, selected node, etcd health and whether HTTP backends themselves lost capacity.

- Optional depth — continue after the core works

Optional depth — leader and peer delay: identify the leader from `endpoint status`, stop that member alone and measure finite write-probe latency. Delay only TCP 2380 in its namespace using Lab 6's filter pattern; test 50 ms before higher delays and delete the qdisc after each test. Effects depend on leader role, quorum path, heartbeat/election settings and pre-vote; no delay value guarantees an election. etcd quorum explanation.

Optional depth — controllers and kubelet: watch the `kube-controller-manager` Lease holder, move that holder's controller-manager manifest outside the directory, delete one lab pod and measure replacement delay; restore the exact manifest. Separately stop an app container through CRI while the API is unavailable to observe local kubelet restart behaviour. These isolate reconciliation from node-local supervision.

Optional depth — API fairness: inspect FlowSchemas and PriorityLevelConfigurations, use a dedicated read-only service account with a finite load of 100 requests and at most five concurrent workers, verify TLS using the kubeconfig CA, and inspect `apiserver_flowcontrol_*` metrics. HTTP 429 is not guaranteed and CPU contention can affect even exempt requests. Remove only the test account and binding afterward; never infer APF from an unverified admin-versus-user timing difference.

Optional depth — stale routing and scale: record proxy mode and backend IPs; save the kube-proxy DaemonSet before a controlled stop/update experiment. Confirm the old proxy processes actually stopped—changing a DaemonSet node selector with rolling-update limits may leave old pods alive. Replace the lab backends, probe the Service and inspect the matching iptables/nftables rules; restore the saved DaemonSet and wait for readiness. Add 50 labelled empty Services first, measure rule-sync metrics, remove them, and increase only after a successful recovery. Empty Services can affect rules differently by proxy mode; do not promise a fixed rule count or benchmark 3,000 by default.

Optional depth — arithmetic: for n members, majority is $\text{floor}(n/2)+1$; tolerated failures are $n - \text{majority}$. Four members tolerate one failure, just like three, but require three votes. This is an availability/cost comparison, not a claim that four is worse in every respect.

Lab 20 — Measure retries under a short slowdown



Theory before you start

- **Concept:** A user intent is one desired operation; an attempt is one execution of it. Retries can improve success during brief failures, but retries at several layers multiply demand. For example, up to two client attempts and two proxy attempts per client attempt can produce four backend attempts. Actual amplification depends on which failures trigger retries and whether earlier attempts succeed.

- **Mechanism:** When a timeout expires, the caller may retry while the previous backend request is still executing or queued. Slow work occupies limited worker slots longer, which increases waiting, triggers more timeouts and adds further attempts. This feedback can outlast the original slowdown; persistent failure under continued load is metastability. The lab compares bounded runs so amplification and recovery can be measured rather than assumed.
- **Read the result:** Calculate backend arrivals per original intent over the same complete run, then compare retries enabled and disabled. Amplification alone does not prove metastability; persistent post-fault failure must be observed.

Study-guide basis: Ch. 1 §1.2.3; Ch. 12 ingress discussion.

Goal: Count extra work caused by retries and compare recovery with retries disabled.

Before you start: Lab 0; ports 8090 and 9001–9002 free; nginx available.

Time: 40–60 min (estimate; downloads excluded).

Core experiment

1. Create a small upstream with a concurrency limit

Save as `~/labs/lab20/upstream.py` (create the directory first):

```
import sys, time, threading
from pathlib import Path
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler
port=int(sys.argv[1]); shed=len(sys.argv)>2 and sys.argv[2]=='shed'
slots=threading.BoundedSemaphore(4)
lock=threading.Lock(); counts={'arrived':0,'done':0,'shed':0}
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        with lock: counts['arrived']+=1
        acquired=slots.acquire(timeout=0.2 if shed else 10)
        if not acquired:
            with lock: counts['shed']+=1
            self.send_response(503); self.end_headers(); return
        try:
            try: delay=float(Path('delay.txt').read_text())
            except (ValueError,FileNotFoundError): delay=0.2
            time.sleep(min(max(delay,0),2))
            with lock: counts['done']+=1
            self.send_response(200); self.end_headers()
            try: self.wfile.write(b'ok\n')
            except (BrokenPipeError,ConnectionResetError): pass
        finally: slots.release()
    def log_message(self,*args): pass
def report():
    while True:
        time.sleep(1)
        with lock: sample=counts.copy()
        print(time.time(),port,sample,flush=True)
```

```
threading.Thread(target=report,daemon=True).start()
ThreadingHTTPServer(('127.0.0.1',port),H).serve_forever()
```

This is a teaching server, not a production service. The finite client below limits offered work; queued requests can outlive their callers. The **shed** mode bounds queue wait before admitting work.

2. Create a two-backend proxy

```
sudo tee /etc/nginx/conf.d/ce-lab20.conf >/dev/null <<'NGX'
upstream ce_lab20 { server 127.0.0.1:9001 max_fails=0; server
127.0.0.1:9002 max_fails=0; }
server {
    listen 127.0.0.1:8090;
    location / {
        proxy_pass http://ce_lab20;
        proxy_connect_timeout 1s;
        proxy_read_timeout 500ms;
        proxy_next_upstream error timeout;
        proxy_next_upstream_tries 2;
    }
}
NGX
sudo nginx -t && sudo systemctl restart nginx
cd ~/labs/lab20
echo 0.2 > delay.txt
python3 upstream.py 9001 > upstream1.log 2>&1 & first=$!
python3 upstream.py 9002 > upstream2.log 2>&1 & second=$!
curl -fsS --max-time 2 http://127.0.0.1:8090/
```

Checkpoint: HTTP 200. **max_fails=0** removes passive ejection as a variable in this comparison. The proxy may attempt up to two backends; this is not unbounded replay of a request. NGINX retry semantics.

3. Send a finite set of user intents

Save as **client.py** in the same folder:

```
import sys,time,urllib.request,urllib.error
from concurrent.futures import ThreadPoolExecutor
retries=int(sys.argv[1]) if len(sys.argv)>1 else 1
def request(i):
    begin=time.monotonic(); outcome='failed'
    for attempt in range(retries+1):
        try:
            with urllib.request.urlopen('http://127.0.0.1:8090/',timeout=2)
as r:
                if r.status==200: outcome='200'; break
        except (urllib.error.URLError,TimeoutError,OSError): pass
    if attempt<retries: time.sleep(0.2)
```

```

print(i,outcome,attempt+1,round(time.monotonic()-begin,3),flush=True)
with ThreadPoolExecutor(max_workers=8) as pool:
    futures=[]
    for i in range(80):
        futures.append(pool.submit(request,i))
        time.sleep(0.25)
    for f in futures: f.result()

```

In terminal 1: `python3 client.py 1 | tee client-retries.log`. In terminal 2, after the first five seconds: `echo 1 > delay.txt; sleep 5; echo 0.2 > delay.txt`.

Prediction: the 1-second work time exceeds the proxy's read timeout, so some user intents create additional backend requests. At most 80 intents, eight active client workers and two attempts per client bound the client. The executor can queue pending intents; record that limitation when interpreting offered rate. Run the delay command from `~/labs/lab20` in terminal 2 too. Each upstream's counters are cumulative, so subtract the pre-test counts (including the setup curl) before calculating amplification.

4. Compare one change

Let all requests finish and backend work drain. Stop and restart the exact upstream PIDs to reset counters. Repeat with `python3 client.py 0`, applying the same five-second slowdown. Compare total arrived requests across both upstreams divided by **80 intents**, plus failures and recovery time.

Do not divide one transient second's arrivals by a nominal rate and call it the whole-run amplification factor. Keep the same load, delay and observation interval.

Cleanup / recovery: `echo 0.2 > delay.txt`; wait for the finite client; `kill "$first" "$second"; wait "$first" "$second" 2>/dev/null || true`; remove only `/etc/nginx/conf.d/ce-lab20.conf`, then `sudo nginx -t && sudo systemctl reload nginx`.

Done when: the journal reports actual amplification and explains which layer retried.

If it differs: inspect HTTP status, upstream counts, worker-queue delay and whether nginx loaded the intended configuration. Low offered load may recover promptly; do not claim metastability unless errors and backlog persist after the fault is removed under continued load.

- Optional depth — continue after the core works

Optional depth: separately set `proxy_next_upstream off`; or start both upstreams with the `shed` argument; or increase the proxy read timeout to `1500ms` and client timeout to 4 seconds. Reset before each comparison. NGINX read timeout is an interval between reads, not a universal whole-request deadline. Retries should be bounded by an end-to-end deadline and restricted to operations safe to repeat. Map a real client→ingress→service→database request's timeouts and retries without changing that real system.

Lab 21 — Run a small team-learning exercise



Theory before you start

- **Concept:** Operational recovery depends on a chain of people, permissions, decisions and documentation. A single specialist can become a knowledge dependency even when the software has redundant replicas. Having a runbook is insufficient if another responder cannot find it, access the tools or interpret its steps. The purpose is to expose weaknesses in that response system, not grade individuals.
- **Mechanism:** A tabletop substitutes a fictional incident for a live outage and assumes the usual specialist is unavailable. The responder explains concrete next actions while a scribe records missing evidence, access or handoffs. A skill matrix helps locate where knowledge is concentrated. Turning one observed obstacle into an owned improvement and rehearsing again tests whether the team's response path actually became more usable.
- **Read the result:** Record where the process stalled and what evidence was missing. A skill matrix identifies dependencies, not personal worth. The useful outcome is one owned improvement followed by a rehearsal; discussion alone does not prove the response works under real conditions.

Study-guide basis: Ch. 13 §§13.2–13.3.

Goal: Find a knowledge dependency without turning the exercise into a judgement of people.

Before you start: No technical cluster needed; agreement from participants before a live exercise.

Time: 30 min design; optional 90 min game day (estimate; downloads excluded).

Core experiment

1. Choose one concrete scenario

Use a tabletop question such as “The build queue is stuck; the usual specialist is unavailable.” List who would detect it, which dashboard they would open and the first reversible recovery action. Do not create a real outage for this core exercise.

2. Build a small skill matrix

Rows: participants. Columns: three operational tasks the team actually owns. Score 0=not yet trained, 1=can do it with help, 2=can do it independently using an available runbook. A lone 2 identifies a training/runbook dependency; it is not evidence of a personal failing. Capture runbook links alongside scores.

3. Conduct a 15-minute tabletop

One facilitator describes the symptom. One responder talks through the runbook. One scribe records missing access, missing evidence and ambiguous instructions. Stop at 15 minutes; record what prevented progress. Compare the predicted path with the actual path.

4. Turn one finding into an action

Write: “Owner **will add/test** by ; **completion evidence** .” Prefer a paired runbook rehearsal or second trained responder. Re-run the tabletop after the action is done.

Cleanup / recovery: end the exercise explicitly; correct any fictional assumptions and return to normal communication. For live games, any real incident ends the game immediately.

Done when: the group has one evidence-backed improvement with an owner and completion check.

If it differs: if there is no gap, record the evidence; do not invent a weakness just to justify the exercise.

- Optional depth — continue after the core works

Optional depth — reliability arithmetic: with independent constant-hazard failures, rate $\lambda = N/MTTF$ gives $P(\text{at least one failure in } t) = 1 - \exp(-\lambda t)$. This is a model, not a measured forecast; shared causes and changing failure rates violate its assumptions. For 400 machines each with MTTF 5 years, use:

```
import math
machines=400; mttf_days=5*365.25
rate=machines/mttf_days
print('daily probability', 1-math.exp(-rate))
print('expected failures in 30 days', 30*rate)
```

MTTF and MTBF are different terms; specify whether repair time is included. Teams × releases × bug probability estimates expected buggy releases under an explicit model, not incident probability or outage duration.

Optional depth — buy-in and game day: write a one-page proposal with the risk assumption, recent incident cost, experiment, stop condition and requested resources. A 90-minute plan: 10 briefing, 10 baseline, 30 experiment/response, 10 recovery, 30 debrief. Assign facilitator, operator, responders, scribe and a stop authority. Use only an agreed lab/non-production target and verify rollback first.

Optional depth — four team games: Staycation exposes dependence on one specialist; Slow Lane delays answers by an agreed interval; Liar, Liar introduces explicitly consented fictional information with an observer and full correction afterward; Inside Job tests a known non-production runbook. Begin with tabletop versions. Never mislead real customers or incident responders. Measure waiting and information flow, then change the system rather than score individuals.

Lab 22 — Design one maintainable resilience check



Theory before you start

- **Concept:** A resilience check starts with a user promise, a measurable service-level indicator and a target. Its hypothesis links one defined fault to an acceptable outcome within a specified window. Baseline health, the observation period and minimum samples make that judgement meaningful. An

injector completing successfully only proves that its action ran; the service outcome requires separate evidence.

- **Mechanism:** Design the checker as another fallible component: it can select the wrong target, stop midway or lose access to the API needed for rollback. Target-count guards, limited duration and a no-fault control bound and clarify the experiment. An independently reachable recovery path removes active faults even if automation fails. Test both a known breach and missing reporting before relying on an unattended check.
- **Read the result:** Assess service behaviour and the checker's reliability separately. A stopped schedule may leave an active fault behind. The design is ready when another engineer can identify the target, success criterion, abort condition and verified recovery path.

Study-guide basis: Ch. 1 §§1.2–1.3; Ch. 2 §2.5; Ch. 11 §11.2; Ch. 13.

Goal: Produce a reviewable experiment with clear evidence and a tested stop path.

Before you start: One completed technical lab and one owned service; core is design-only.

Time: 45–60 min design; implementation scheduled separately (estimate; downloads excluded).

Core experiment

1. Choose one promise

Example: "A new disposable build-agent pod becomes usable within 60 seconds." Choose a promise your users experience, one target environment and one failure mode. The number is a proposed target, not a universal SLO.

2. Fill in this experiment card

```
Service / owner / environment:
User promise and measurement source:
Baseline window and minimum sample count:
Hypothesis, threshold and observation window:
Injection: exact selector, maximum matched objects, fault and duration:
Preflight: expected target count, healthy baseline and capacity check:
Abort: observable condition and person/process allowed to stop:
Rollback: exact command and independent fallback if API access fails:
Recovery: baseline signal and acceptable recovery time:
Evidence: per-run outcome, duration, timestamp, logs and versions:
Review: owner, reviewer and next review date:
```

3. Review the failure of the experiment itself

Ask: what if the selector matches zero or 100 objects; what if the injector dies; what if the API is unavailable; what if results stop arriving? Reject runs with an unexpected target count. A successful automation job is not automatically a successful user outcome.

4. Define a staged implementation decision

Start with a no-fault control, then one supervised experiment in a disposable environment. Only after that succeeds, propose an explicitly approved non-production pilot. Use a least-privilege identity and an enforced maximum duration. Stopping a CronJob or schedule prevents future injections but does not necessarily remove an active fault; rollback must handle both.

5. Write the acceptance evidence

The design is complete when a reviewer can identify the target, expected result, stop command and recovery probe without asking the author. Implementation acceptance additionally needs a successful rollback drill, a deliberately failed SLO check, a missing-results alert test and retained per-run evidence.

Cleanup / recovery: core design makes no system change. During implementation, the card's exact rollback and independently reachable fallback are mandatory, followed by the recovery probe.

Done when: the card and review findings are complete; deployment remains a separate, authorized activity.

If it differs: an unmeasurable promise, unknown target count or untested rollback is a design finding—resolve it before a live pilot.

- Optional depth — continue after the core works

Optional depth — continuous operation: emit success, duration and last-run timestamp, plus durable run history. Monitor SLO failures and missing/scrape-failed results separately. Review cost, false alerts and injector coverage after upgrades. Two clean weeks are a useful proposed observation window, not proof that promotion is safe; choose sample size and coverage based on risk and review findings. A real failure means the experiment found a weakness, not that the experiment should be suppressed. Retire or repair checks whose results nobody owns.

Coverage and review notes

Book-to-lab map: Ch. 1 → 1, 20, 22; Ch. 2 → 2–3; Ch. 3 → 4; Ch. 4 → 5–6; Ch. 5 → 7–9, 11; Ch. 6 → 10–11; Ch. 7 → 12; Ch. 8 → 13; Ch. 9 → 14; Ch. 10 → 15–16; Ch. 11 → 17–18; Ch. 12 → 18–20; Ch. 13 → 21–22. Setup and the journal support the appendices; this mapping does not claim every appendix item is a hands-on lab.

What the refactor preserves: hypothesis and baseline, injection evidence, recovery, mechanism, failure interpretation, chapter coverage and optional deeper investigation. The JVM and browser core paths use smaller self-contained fixtures; historical application reproductions remain optional. The disk exercise uses a bounded shared filesystem rather than filling the VM root disk. The capstone starts with a design rather than directing immediate changes to a real platform.

Review scope: all 23 labs reviewed; shell snippets parsed, Python snippets syntax-checked, YAML parsed, browser JavaScript syntax-checked and the book's C server syntax-checked. The isolated JVM fixture was compiled and run locally: ten baseline output lines, zero injected normal-output lines and ten caught exceptions. It was tested with Java 17 class files on a newer local JDK, not on the target Ubuntu VM. Full Ubuntu/Docker/Kubernetes runtime validation is not claimed. Expected timings and performance gains must be measured on the lab VM.

Primary references: book example source, cgroup v2 interfaces, Docker resource limits, systemd service semantics, strace project and manual. Topic-specific references appear beside the relevant labs.